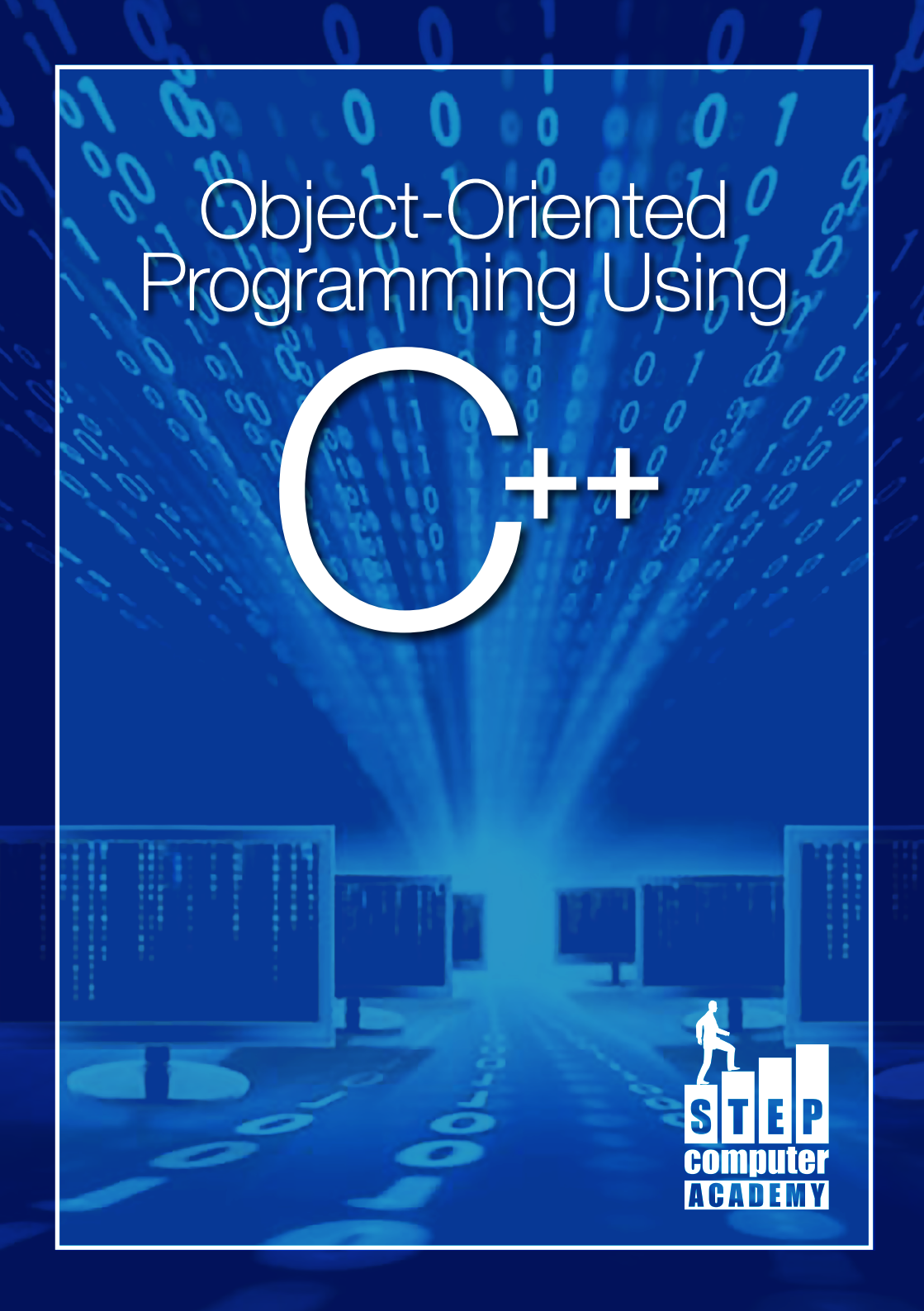
Object-Oriented Programming Using C++



Lesson 5

Class Templates. The ‘string’ Class

Contents

Lesson materials are attached to this PDF file. In order to get access to the materials, open the lesson in Adobe Acrobat Reader.

1. Static Polymorphism and Templates as a Special Type

In programming languages and type theory, polymorphism is the provision of a single interface to entities of different types or the use of a single symbol to represent multiple different types.

Wikipedia

1.1. The Concept of Polymorphism

Polymorphism. Literal translation of the original meaning (ancient Greek) is "many forms".

In its most general form, polymorphism means:

- performance of the same function by different forms (for example, watches: sandglasses, mechanical, electronic are very different, but all do the same — measure time);
- union of several forms in one essence/creature (the mythical Minotaur — a man-bull).

In programming, polymorphism is understood in a first sense: polymorphic code processes different types of values while running a program.

A clear example of such polymorphism is the example with clocks mentioned above: various watch works determine and show the time, although they receive and process different types of signals (flow of sand, pendulum motion, electromagnetic oscillations).

Polymorphism in a program can be expressed differently. Let's trace the sequential process of this feature — from complete absence to the maximum possible value.

Imagine that we needed to develop a function for sorting an array of integers. Developed, debugged… Function prototype will look as follows:

```
void sort(int* intArray, int size);
```

And then, we needed to develop also a floating-point function for sorting an array of integers. Without supporting polymorphism in programming languages, we would have to come up with another function name because the function is different and the parameter types are not corresponding:

```
void sortD(double* dArray, int size);
```

After then, the function for sorting strings was needed as well:

```
void sortS(string* sArray, int size);
```

It turns out impractical somehow: several functions with the same purpose and almost the same code are called differently, and we ought to remember where we should apply each of them. It is clear that these difficulties are easy to deal with in this particular case. But imagine a completely ordinary project with dozens of developers, thousands of functions, and many tasks where the functions are only slightly different, similar to the example above.

This is where polymorphism comes to the rescue — at an early stage as a simple function overloading.

```
void sort(int* array, int size);
void sort(double* array, int size);
void sort(string* array, int size);
```

Using function overloading, the program will look better, the number of concepts and names is reduced. Now we only have one function name, one function assignment, one call option, one interface, and several implementation options. With such an approach, it becomes easier to write code and modify it if necessary.

1.2. Static and Dynamic Polymorphism

You need to understand that polymorphism is not a unique feature of C++, and it is not only one of the major OOP principles. Polymorphism is an essential concept of the theory and practice of software development. Polymorphism is used extensively in different programming languages and interfaces.

There are two types of polymorphism in a programming environment that differ significantly from each other:

- run time polymorphism (or dynamic polymorphism);
- compile-time polymorphism (or static polymorphism).

Dynamic polymorphism is implemented at runtime. It is the use of virtual functions, which are handled through pointers. We will get familiar with virtual functions later.

Static polymorphism is implemented at compile time. C++ provides three kinds of static polymorphism:

- function overloading;
- operator overloading;
- templates.

Exists function templates and class templates.

The function template is like a writing algorithm that can be performed with different data types. The data type is

a special type template parameter. Substituting various data types into this template, we will get a set of different functions with the same algorithm and different data types.

A class template is a generic description of a class, which is like a function template — it makes sense for the different data types used to represent the fields and functions of the class.

1.3. Function Templates

So, we have three twin functions that sort arrays of different types in the same way.

```
void sort(int* array, int size);
void sort(double* array, int size);
void sort(string* array, int size);
```

If their codes differed, then we would have to leave it at that. But they are identical, excepting the variables that differ in data type. For this case, C++ provides function templates. We will write only one function (one function template). We will try to provide the possibility of using different data types by setting special type template parameters.

Undoubtedly, one function text is much more practical than three, especially in terms of program maintenance. Code duplication is like a nightmare for a programmer. Programs are modifiable, and while you are changing one part of a code, it's easy to forget about implementing similar changes in another part of the code. The result is an error that will take time and resources to find and fix it.

Function template generally starts with a 'template' keyword followed by a list of type parameters in angle

brackets. Type-parameters are specified with 'typename' or 'class' keywords (you can use both, there is no difference between them).

Then, defining a function is followed by, where the titles of type parameters are used instead of specifying definite types for variables with modifiable types of data.

```
template < template-parameter-list > function-declaration
```

For instance, function template for calculating the square number:

```
template<class T>
T square(T number)
{
    T result = number * number;
    return result;
}
```

T class here is a defining type parameter, where T is a mutable data type. A function can square integers and floating numbers.

So, let's continue developing polymorphism and write a template of a 'sort' function mentioned earlier.

```
template <class T>
void sort(T array[], size_t size)
{
    for (int k = size – 1; k > 0; k--)
    {
        for (int j = 0; j < k; j++)
        {
            if (array[j] > array[j + 1])
            {
```

```
                swap(array[j], array[j + 1]);
            }
        }
    }
}
```

Function template development consists of the following steps:

- a function is created for a specific type (in our example is a bubble sorting for the int type);
- a list of template parameters is added before the function header: "template" keyword and a list of template type parameters in angle brackets (in our example is template <class T>);
- the type name (int) in the function implementation is replaced by the type parameter (T) name.

So, we have reached the top level of polymorphism, as we have available only one code, solving the same task for several different data sets.

Similarly, we will develop a function template for displaying an array on the screen.

```
template<class T>
void display(T array[], size_t size)
{
    for (int i = 0; i < size; i++)
    {
        cout << array[i] << " ";
    }
    cout << endl;
}
```

Now we can use the created templates for sorting and displaying arrays of different types.

The type size_t used in the example is an unsigned integer type commonly used for loop counters, array indexing, and storing object sizes.

Calling (applying) a template function is performed in the same way as calling a regular function: the name of the function and the list of parameter values in parentheses are specified.

But at the same time, you need to specify the values of the type template parameters. These values are written in angle brackets between the function name and the parameter list like this: sort<int>(intArray, size). Where 'sort', as a function call, is used to process integers array.

Let's look at the code with the created templates mentioned above: (The program code is in the folder *Lesson05\les05_01*).

```
int main()
{
    cout << "Sort Template" << endl << endl;

    int intArray[]{ 1, 3, 7, -4, -2, 4 };
    int size = sizeof(intArray) / sizeof(int);
    cout << "Original int Array : ";
    display<int>(intArray, size);
    sort<int>(intArray, size);
    cout << "Sorted   int Array : ";
    display<int>(intArray, size);

    char chrArray[]{ 'o', 't', 't', 'f', 'f', 's', 's',
                     'e', 'n' };
    size = sizeof(chrArray) / sizeof(char);
    cout << "Original chr Array : ";
```

```
    display<char>(chrArray, size);
    sort<char>(chrArray, size);
    cout << "Sorted   chr Array : ";
    display<char>(chrArray, size);
    string strArray[]{ "one", "two", "three", "four", "five"};
    size = sizeof(strArray) / sizeof(string);
    cout << "Original str Array : ";
    display<string>(strArray, size);
    sort<string>(strArray, size);
    cout << "Sorted   str Array : ";
    display<string>(strArray, size);
    _getch();
    return 0;
}
```

Besides int and char built-in data types, the above example uses a 'string' type, which you will get to know in detail soon afterward. 'string' is a class of the C++ standard library, which is used for working with the text data. The purpose of the 'string' is the same as a character array has (char[], char*), but the 'string' type has more advanced tools to work with text.

The program result is shown in Figure 1.

```
Sort Template

Original int Array : 1 3 7 -4 -2 4
Sorted   int Array : -4 -2 1 3 4 7
Original chr Array : o t t f f s s e n
Sorted   chr Array : e f f n o s s t t
Original str Array : one two three four five
Sorted   str Array : five four one three two
```

Figure 1. Using 'sort' function template for sorting arrays of different types

Specifying the value of the type parameter is optional, as it is shown as follows:

```
display<int>(intArray, size);
sort<int>(intArray, size);
```

Or, it can be written like this:

```
display(intArray, size);
sort(intArray, size);
```

The type parameter compiler (intArray — int) determines for what type you are going to generate a function from a template. But in some cases, ambiguity may arise, and then it will be necessary to indicate a type parameter.

The function template has an important feature: until there is no function call, a template does not compile.

Template is compiled for each type parameter, for which it was called (the above example compiles 3 versions of the 'sort' function: for 'int', 'string', and 'char').

If the function template is applied (called) in several files of the project, then you should place the body of the function template in the title of the file (in **.h* file, but not in **.cpp*-file).

If we use the function template as a regular function, that is, we place the body of the function template in **.cpp* file, and the prototype of the function template — in the calling function file (the program code is in the folder *Lesson05\les05_01a*), then the compilation of the template call will proceed without occasions. Still, its arranging will end with error messages, as shown in Figure 2:

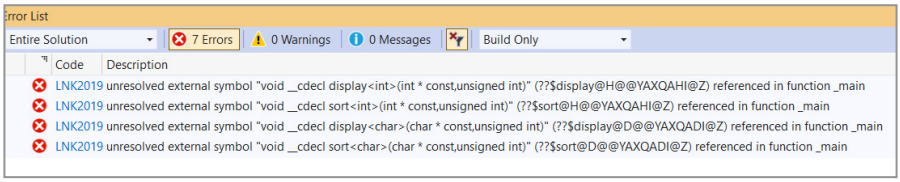


*Figure 2. Program compilation error with placing the body of a function template in the *.cpp file*

However, in fact:

- function template is only compiled when calling;
- body of a function template, placed in another file, is not obvious for a compiler when compiling a call of a function template, and a compiler does not compile function template body when calling;
- a compiler does not compile the template without calling.

Therefore, the function template is not compiled in this case, and the linker reports that there is no called function.

Here is another example of a function template.

The template of the getValue function for input the different data types from the keyboard:

```
template <class T>
void getValue(string prompt, T& value)
{
    cout << prompt;
    cin >> value;

    while (cin.fail())
    {
        cin.clear();
        cin.ignore(32767, '\n');
```

```
        cout << "Error... try again" << endl;
        cout << prompt;
        cin >> value;
    }

    string endLine;
    getline(cin, endLine);
}
```

The following example uses the input stream handling library functions:

- cin.fail() function returns true if input meets the error;
- cin.clear() function restores the input stream from "wrong" condition the "working";
- cin.ignore() function clears the input buffer from the remaining not entered characters.

All this is unnoted in a loop and forces a user to repeat input in case of input error until their error stops showing up. Exiting the loop (or bypassing it) is going only with the correct values input.

This function is focused on inputting a single value and pressing 'Enter' ("Input") after inputting. When reading the value (cin >> value;), "Enter" ('\n' — end of a string) stays in the input stream. And you will need the last two strings of the template to extract this character from the stream.

```
string endLine;
getline(cin, endLine);
```

The getline(cin, endLine) function reads all data up to the nearest '\n' character, including the symbol, into 'endLine'

variable of a 'string' type. Thus, the input stream will be cleared and ready to enter the next values.

Using this template, it is convenient to organize the input of different data types, for example:

```
int number;
getValue("Enter number: ", number);

string name;
getValue("Enter name: ", name);
```

Polymorphism usage in all its varieties makes programs understandable and easier to maintain. In addition to this, applying C++ templates leads to shorter and easier modifiable programs.

Templates are a bit difficult to learn. However, our benefits from their usage outweigh all the difficulties. You should also remember that templates are widely used in the standard C++ libraries, and it is hard to see writing modern programs without intensive use of these libraries.

2. Class Templates

Generic programming is a style of computer programming in which algorithms are written in terms of types to-be-specified-later that are then instantiated when needed for specific types provided as parameters.

Wikipedia

2.1. Class Templates

Generic programming is also called generalized programming. In C++, generic programming is based on function and class templates.

Class templates allow you to create a description of an abstract class. While using the template it is expanded by the compiler into definite classes that implement the specified functions for various data types.

Declaring class template is similar to declaring function templates:

```
template < template-parameter-list > class-declaration
```

For example:

```
template<class T>

class Array
{
    T* array;
    size_t size;
. . .
};
```

Class template starts with 'template' followed by a list of class template parameters in angle brackets.

Class template parameters can be type parameters, regular type parameters, and template parameters.

And, class fields, constructors, and methods are placed, as usual, in the class body. But one difference is that instead of pointing to a specific data type, the name of the template type parameter is used (T) in some parts of the code, for example:

```
T getItem(size_t index) const
void setItem(size_t index, T value)
```

Let's consider an example (the program code is in the folder *Lesson05\les05_02*):

```
#include <iostream>
#include <conio.h>
using namespace std;

void stopProgram(string message)
{
    cout << message << endl;
    cout << "Press any key to exit program" << endl;
    _getch();
    exit(1);
}

template<class T>
class Array
{
    static const size_t size{ 5 };
    T arr[size];

public:
```

```
Array()
{
    for (int i = 0; i < size; i++)
    {
        arr[i] = T();
    }
}

int getSize() const
{
    return size;
}

T getItem(size_t index) const
{
    if (index >= 0 && index < size)
    {
        return arr[index];
    }
    else
    {
        stopProgram("Index is out of range!");
    }
}

void setItem(size_t index, T value)
{
    if (index >= 0 && index < size)
    {
        arr[index] = value;
    }
    else
    {
        stopProgram("Index is out of range!");
    }
}
```

```
    void display()
    {
        for (int i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void sort()
    {
        for (int k = size - 1; k > 0; k--)
        {
            for (int j = 0; j < k; j++)
            {
                if (arr[j] > arr[j + 1])
                {
                    swap(arr[j], arr[j + 1]);
                }
            }
        }
    }
};

int main()
{
    cout << "Class Array" << endl << endl;

    Array<int> intArray;
    cout << "int Array initialization:" << endl;
    intArray.display();
    int size = intArray.getSize();
    for (int i = size; i > 0; i--)
    {
        intArray.setItem(size - i, i);
    }
    cout << endl << "int Array after assignment:" << endl;
```

```
    intArray.display();
    intArray.sort();
    cout << endl << "int Array after ordering:" << endl;
    intArray.display();
    cout << endl;

    Array<string> strArray;
    cout << "str Array initialization:" << endl;
    strArray.display();
    strArray.setItem(0, "two");
    strArray.setItem(1, "seven");
    strArray.setItem(2, "zero");
    strArray.setItem(3, "four");
    strArray.setItem(4, "one");
    cout << endl << "str Array after assignment:" << endl;
    strArray.display();
    strArray.sort();
    cout << endl << "str Array after ordering:" << endl;
    strArray.display();

    cout << endl << "Press any key to exit program" << endl;
    _getch();
    return 0;
}
```

The class template code and its usage are located in one file for simplicity in this program. Obviously, the template code must precede the 'main' function where its methods are called.

Comments on the class template code:

- Template of the Array class presents, for simplicity of the example, a static array template whose size is given by a static constant size{ 5 }.
- Assignment arr[i] = T() causes array fields get values of T type by default. 0 is for numeric types, an empty string

is for 'string', and for user-defined types is a value that sets the default constructor for this type (constructor without parameters).

- Class member functions provide access to the array elements, sorting, and array outputting.
- When accessing elements of the T arr[size] array, the index is checked.
- The stopProgram function is used for program crashes in case of error.

Using a class template is quite simple. But be sure to specify the template type parameter in angle brackets when calling the class constructor:

```
Array<int> intArray;
Array<string> strArray;
```

And call the necessary methods as a next step.

```
intArray.display();
int size = intArray.getSize();
strArray.sort();
```

In the given program, we create two objects of different types based on the template of the Array class:

- integer array intArray;
- string array strArray;

and using the template methods, we will perform the same actions with them: assigning values, sorting, outputting/displaying on the screen.

The program result is shown in Figure 3.

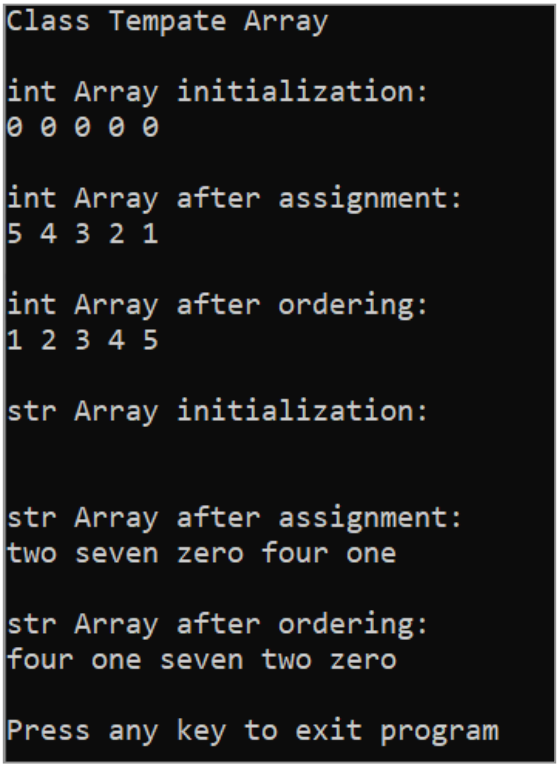


Figure 3. Using a template of the Array class for working on the arrays of different types

In the above example, we used the easiest way to write the code for the class template; all functions were set in the class body. But it's not regularly convenient. For example, some occasions appear when somewhat separate prototypes of the class methods from their implementation.

Method implementation outside the class body requires a detailed description of the types used in the method. And upon that, this requires multiple references to the template.

Let us have a template of a 'Sample' class with 'square' method:

```
template<class T>

class Sample
{
    ...
    T square(T number);
}
```

The 'square' method implementation outside the class will be like this:

```
template<class T>

T Sample<T>::square(T number)
{
    T result = number * number;
    return result;
}
```

Firstly, you must indicate a template before the title of the function:

```
template<class T>
```

Secondly, you need to specify that this function belongs to the template of the Sample class with T type parameter:

```
Sample<T>::square
```

Let's give an example with the template of the Array class, which will be transformed in such a way (the program code is in the folder *Lesson05\les05_03)*:

The main modifications compared to the previous version:

- the code of the Array class template was placed into a separate *array.h* header file;
- this header file is in the *les05_03.cpp* file, where the Array class is used;

```
#include <iostream>
#include <conio.h>
#include "array.h"
using namespace std;

void stopProgram(string message)
{
    cout << message << endl;
    cout << "Press any key to exit program" << endl;
    _getch();
    exit(1);
}

int main()
{
    cout << "Class Template Array" << endl << endl;

    Array<int> intArray;
.   .   .
```

- in the *array.h* file the class body includes only the fields and prototypes of the class methods:

```
#pragma once
#include <iostream>

void stopProgram(std::string message);

template<class T>
```

```
class Array
{
    static const size_t size{ 5 };
    T arr[size];

public:
    Array();
    int getSize() const;
    T getItem(size_t index) const;
    void setItem(size_t index, T value);
    void display();
    void sort();
};
```

- the implementation of the methods is not included in the class body:

```
template<class T>
Array<T>::Array()
{
    for (int i = 0; i < size; i++)
    {
        arr[i] = T();
    }
}

template<class T>
int Array<T>::getSize() const
{
    return size;
}

template<class T>
T Array<T>::getItem(size_t index) const
{
```

```
    if (index >= 0 && index < size)
    {
        return arr[index];
    }
    else
    {
        stopProgram("Index is out of range!");
    }
}
. . .
```

You should remember that template<class T> and Array<T>:: must be obligatorily presented in the header of a function implemented outside the class body. We know that function template code should be placed in a header file (*.*h*). The same applies to class templates. All class template code (including class declaration and implementation of its methods) is recommended to be placed in one header file.

If the placement of the whole code will do the *.*h* file too massive or disorganized, then the alternative would be placing it into the file *.*inl* ('.*inl*' means "*inline*" = "built-in"), and then connecting *.*inl* to the *.*h* file. It will provide us with the same result as putting all the code in a header file, but the code will be separated in this way.

If the implementation of the class template functions will be put in a separate *.*cpp* file, then the same mistakes will appear as a result that when placing the function template in the *.*cpp* file. It means that the program compilation will succeed, but the arranging will end with problematic errors because the functions of the class template will not be compiled.

Template parameters of a class (inside the angle brackets) can not only type parameters (<class T>), but also parameters of standard types. Such parameters are called as non-type parameters.

Let's consider an example (the program code is in the folder *Lesson05\les05_04*).

```
#include <iostream>
#include <conio.h>
using namespace std;

template <class T, int size> // size is a non-type parameter
                             // in the class template
class StatArray
{
private:
    // non-type size parameter defines the size
    // of the static array
    T m_array[size];

public:
    T* getArray();
    T& operator[](int index)
    {
        return m_array[index];
    }
};

// defining a method outside the class body with
// a non-type parameter
template <class T, int size>
T* StatArray<T, size>::getArray()
{
    return m_array;
}
```

```
int main()
{
    cout << "Class Template with Non-type parameters"
         << endl << endl;

    // integer array of 10 elements
    const int size = 10;
    StatArray<int, size> intArray;
    for (int count = 0; count < size; ++count)
        intArray[count] = count;

    // array elements in the reverse order
    for (int count = size – 1; count >= 0; --count)
        std::cout << intArray[count] << " ";
    std::cout << '\n';

    // a double type array of 8 elements
    StatArray<double, 8> doubleArray;

    for (int count = 0; count < 8; ++count)
        doubleArray[count] = 5.5 + 0.1 * count;

    for (int count = 0; count < 8; ++count)
        std::cout << doubleArray[count] << ' ';

    _getch();
    return 0;
}
```

The program result is shown in Figure 4.

```
Class Tempate with Non-type parameters

9 8 7 6 5 4 3 2 1 0
5.5 5.6 5.7 5.8 5.9 6 6.1 6.2 _
```

Figure 4. Applying the non-type parameter of the class template

In this example, the template parameter 'size' (of the int type) is causing the size of the class's allocated static array.

```
template <class T, int size>

class StatArray
{
private:
    T m_array[size];
```

This and other parameters must take part in every class method definition outside the class body.

```
template <class T, int size>

T* StatArray<T, size>::getArray()
{
    return m_array;
}
```

When creating an object of the non-type class, parameter must be set with a constant value.

```
const int size = 10;
StatArray<int, size> intArray;

StatArray<double, 8> doubleArray;
```

In general occasions, a constant expression can be as a value of a non-type parameter and be of such kind:

- integer value or enumeration;
- a pointer or a reference to a class object;
- a pointer or a reference to a function;
- a pointer or a reference to a class method.

A class template can also be a parameter of a class template! Consider an example (the program code is in the folder *Lesson05\les05_05*):

```
#include <iostream>
#include <vector>
#include <conio.h>
using namespace std;

template<class T>
struct Point
{
    T x;
    T y;
    void display()
    {
        cout << "(" << x << "," << y << ")";
    }
};

template<class T>
struct Array
{
    vector<T> data;

    void Add(T item)
    {
        data.push_back(item);
    }

    void display()
    {
        for(auto var:data)
        cout << var << " ";
    }
};
```

```
template<template<class> class T, class T1>
struct Some
{
    T<T1> data; // data variable is created,
                // its type will be the template of T type,
                // receiving T1 type parameter

    void Add(T1 item)
    {
        data.Add(item);
    }

    void display()
    {
        data.display();
        cout << endl;
    }
};

int main()
{
    // Point structure with integers 'x' and 'y'
    Some<Point, int> intPoint;
    intPoint.data.x = 1;
    intPoint.data.y = 2;
    cout << "Some: struct Point with int  x, y  : ";
    intPoint.display();

    // Point structure with floating 'x' and 'y'
    Some<Point, double> doublePoint;
    doublePoint.data.x = 10.01;
    doublePoint.data.y = 0.02;
    cout << "Some: struct Point with double x,y : ";
    doublePoint.display();

    // integer array (vector)
    Some<Array, int> intArray;
```

```
    intArray.Add(1);
    intArray.Add(3);
    intArray.Add(5);
    cout << "Some: array (vector) with int items: ";
    intArray.display();

    _getch();
    return 0;
}
```

The program includes:

- the Point class template (point's coordinates) with T type parameter (as a type of coordinates):

```
template<class T>
struct Point
{
    T x;
    T y;
. . .
```

- the Array class template with T type parameter (as an array element type):

```
template<class T>
struct Array
{
    vector<T> data;
. . .
```

- the Some (something) class template with the class template parameter <template>class T and T1 type parameter for this template:

```
template<template<class> class T, class T1>
struct Some
{
    T<T1> data;
. . .
```

Based on the Some template, we can create:

```
// Point structure with integers 'x' and 'y'
Some<Point, int> intPoint;
. . .

// Point structure with floating 'x' and 'y'
Some<Point, double> doublePoint;
. . .

// integer array
Some<Array, int> intArray;
```

The shown example has an educational meaning only, not practical. But it shows you obvious possibilities of applying the C++ templates.

The program result is shown in Figure 5.

```
Some: struct Point with int  x, y  : (1,2)
Some: struct Point with double x,y : (10.01,0.02)
Some: array (vector) with int items: 1 3 5
```

Figure 5. Using the template parameters in the class template

Class template parameters, like the standard parameters, can be default parameters.

Let's define the Array class template as follows:

```
template <class T = int, int size = 10>
class Array
```

Then the following template usage

```
Array <> intArray;
```

will be equivalent to

```
Array <int, 10> intArray;
```

and will result in an array of 10 integers.

2.2. Template Requirements for the Type Parameters in Use. Standard Template Library (STL)

The template code can have certain requirements to the used types parameters.

Let's consider a simple example (the program code is in the folder *Lesson05\les05_051*):

```
#include <iostream>
#include <conio.h>
using namespace std;

template< typename T >
void Display(T value)
{
    cout << value << endl;
}

template< typename T >
T Sum(T value1, T value2)
{
```

```
    return value1 + value2;
}

struct Point
{
    int x;
    int y;
};

int main()
{
    cout << "Template Requirements" << endl;
    int n1 = 1;
    int n2 = 2;
    int n3 = Sum(n1, n2);
    Display(n1);
    Display(n3);
    double d1 = 1.0;
    double d2 = 2.0;
    double d3 = Sum(d1, d2);
    Display(d1);
    Display(d3);
    Point p1{ 1,1 };
    Point p2{ 2,2 };
    //Point p3 = Sum(p1, p2);
    //Display(p1);
    //Display(p3);

    _getch();
    return 0;
}
```

In the given example, there are:

- function template void Display(T value), which displays the parameter value on the screen;

- function template T Sum(T value1, T value2), which returns the sum of the values of its two parameters;
- the 'Point' structure, representing the points coordinates (x, y) on the plane;
- the 'main' function, testing the 'Display' and 'Sum' function templates for type parameters 'int', 'double', and 'Point'.

If you uncomment the commented strings

```
// Point p3 = Sum(p1, p2);
// Display(p1);
// Display(p3);
```

the compiler will output the error messages

```
Error C2676 binary '+': 'T' does not define this operator
      or a conversion to a type acceptable
      to the predefined operator
Error C2679 binary '<<': no operator found which takes
      a right-hand operand of type 'T' (or there is
      no acceptable conversion)
```

If the strings are commented, the program will work correctly (see Figure 6)

```
Template Requirements
1
3
1
3
```

Figure 6. Using function templates for different type parameters

Therefore, the cause of errors is not in function templates, but in the fact that these templates require to stick to the certain rules of processing from a type parameter acting with them (refers to the 'Point'):

- the used type must have defined (overloaded)operator '+' ('Sum' for function template);
- the used type must have defined (overloaded) the operator '<<' ('Display' for function template).

Add the necessary overloading to the 'Point' structure (the program code is in the folder *Lesson05\les05_051a*):

```
. . .
struct Point
{
    int x;
    int y;

    Point(int x = 0, int y = 0) : x{x}, y{y}
    {}

    Point operator+(Point n)
    {
        return Point(this->x + n.x, this->y + n.y);
    }

    friend ostream& operator<<(ostream& output, const Point& p)
    {
        output << "(" << p.x << "," << p.y << ")";
        return output;
    }
};

int main()
{
```

```
    cout << "Template Requirements" << endl;
    int n1 = 1;
    int n2 = 2;
    int n3 = Sum(n1, n2);

    Display(n1);
    Display(n3);

    double d1 = 1.0;
    double d2 = 2.0;
    double d3 = Sum(d1, d2);

    Display(d1);
    Display(d3);

    Point p1{ 1,1 };
    Point p2{ 2,2 };
    Point p3 = Sum(p1, p2);

    Display(p1);
    Display(p3);

    _getch();
    return 0;
}
```

The program result is shown in Figure 7.

```
Template Requirements
1
3
1
3
(1,1)
(3,3)
```

Figure 7. Using function templates for type parameters with overloaded operators

As we see, the requirements of function templates for type parameters are implemented, and the program works correctly for all data types used.

Class templates tend to be even more sensitive to the type parameters they use. In addition to the operator overloading (as with function templates), class templates usually require the following items in the used types:

- default constructor;
- copy constructor;
- assignment operator;
- comparison operators;
- input/output operators.

Some templates may also require:

- destructor;
- move constructor.

Other (standard) methods of overloading may also be necessary, which are used in the template code like method overloading of min, max, swap, etc.

Consider the next example. For this, we will take template<class T> class Array from the program *Lesson05\les05_03* and try to apply the Point structure as a type T parameter (the program code is in the folder *Lesson05\les05_052*).

```
struct Point
{
    int x;
    int y;
};
```

```
int main()
{
    cout << "Class Template Array (Points)" << endl << endl;

    Array<Point> pointArray;
    cout << "Point Array initialization:" << endl;
    pointArray.display();
    int size = pointArray.getSize();

    for (int i = size; i > 0; i--)
    {
        pointArray.setItem(size – i, Point{ i,i });
    }

    cout << endl << "Point Array after assignment:" << endl;

    pointArray.display();
    pointArray.sort();

    cout << endl << "Point Array after ordering:" << endl;
    pointArray.display();

    cout << endl;
    cout << endl << "Press any key to exit program" << endl;

    _getch();

    return 0;

}
```

And we will get such errors when compiling:

```
C2679  binary '<<': no operator found which takes
       a right-hand operand of type 'T' (or there is
       no acceptable conversion)
```

```
C2676  binary '>': 'T' does not define this operator or
       a conversion to a type acceptable to the predefined
       operator
```

The compiler reports that the output operator << and the comparison operator > are not defined in the type T parameter (the struct Point).

Define the operators needed for applying the template template<class T> class Array in the struct Point class (the program code is in the folder *Lesson05\les05_052a*)

```
struct Point
{
    int x;
    int y;

    friend bool operator>(const Point& p1,
                          const Point& p2)
    {
        double d1 = sqrt(p1.x * p1.x + p1.y * p1.y);
        double d2 = sqrt(p2.x * p2.x + p2.y * p2.y);
        return d1 – d2;
    }

    friend std::ostream& operator<<(std::ostream& output,
                                    const Point& p)
    {
        output << "(" << p.x << "," << p.y << ")";
        return output;
    }
};
```

Now the program compiles and runs correctly (see Figure 8).

```
Class Template Array (Points)

Point Array initialization:
(0,0) (0,0) (0,0) (0,0) (0,0)

Point Array after assignment:
(5,5) (4,4) (3,3) (2,2) (1,1)

Point Array after ordering:
(1,1) (2,2) (3,3) (4,4) (5,5)


Press any key to exit program
```

Figure 8. Using a class template for type parameters with overloaded operators

Templates are a very effective and convenient tool in programming, and therefore they are widely used in C++. Many ready-to-use templates (classes and functions) are contained in the standard template library (STL). This library is an integral part of C++. STL contains a set of classes of the containers to produce arrays, linked lists, trees, sets of values, sets of value-key pairs, etc. STL also contains the processing function of elements of these classes (for example, finding the maximum, sum, or single element) and other helper functions.

One of the most used STL templates is a 'vector' (one-dimensional array) — a dynamic array template. Vector is declared like this:

```
vector<data_type> variable_name;
```

Operator, for instance

```
vector <int> intArray;
```

declares a dynamic array of integers. You can use vector functions to add data to this array:

- push_back — adding a value after the last element of the vector;
- insert — value insertion before the specified element of the vector.

These functions add new elements and increase the vector's size. Access to the vector's elements is implemented in the same way as for the elements of a standard array:

```
intArray[2] = 10
```

The 'erase' function is used for removing. It deletes specified vector elements and reduces the size of the vector.

So, the vector is a standard array whose size can be changed by adding and removing elements while running the program.

2.3. Function Template Specialization

We get the same implementation (execution of the same algorithm) when using function templates for different data types. Sometimes it does not work and, suddenly, it turns out that the algorithm should be different for some data types.

Let's consider an example (the program code is in the folder *Lesson05\les05_06*):

```
#include <iostream>
#include <conio.h>
using namespace std;

template <class T>
```

```
T Max(T t1, T t2)
{
    return (t1 > t2 ? t1 : t2);
}

int main()
{
    int i1 = 10;
    int i2 = 20;
    double d1 = 10.0;
    double d2 = 20.0;
    string s1 = "hi";
    string s2 = "bye";
    const char* p1 = "hi";
    const char* p2 = "bye";
    cout << Max(i1, i2) << endl;
    cout << Max(d1, d2) << endl;
    cout << Max(s1, s2) << endl;
    cout << Max(p1, p2) << endl;

    _getch();
    return 0;
}
```

Using function template T Max(T t1, T t2), it must return the greater of its two parameter values. Check it (see Figure 9):

Figure 9. Using 'Max' function template for different data types

The function works correctly with 'int', 'double', 'string' types. But the result of comparing the data of the const char* type is incorrect: the function returns "*bye*" instead of the correct "*hi*". The thing is, the data address values are compared in this example, but not the data values, pointer values to the character arrays, and so on.

And it appears that the const char* needs a different implementation of the function template. And C ++ has such an opportunity. This method is called a function template specialization. Template specialization exists in order to modify the condition of the general template to the specifics of individual data types.

Function template specialization is a definition where the 'template' keyword follows by empty angle brackets <>, followed by the definition of a specialized template which includes the name of the template, the arguments for its specializing, the list of function parameters, and its body.

```
template <>
returnDataType functionName <dataType> (argumentList)
{
    functionBody
}
```

For the 'Max' function template described above, specialization for the const char* data type will be like this (the program code is in the folder *Lesson05\les05_06a*):

```
template <>
const char* Max<const char*>(const char* t1, const char* t2)
{
    return (strcmp(t1, t2) > 0 ? t1 : t2);
}
```

The template specialization must be placed after the general template.

```
template <class T>
T Max(T t1, T t2)
{
    return (t1 > t2 ? t1 : t2);
}

template <>
const char* Max<const char*>(const char* t1,
                             const char* t2)
{
    return (strcmp(t1, t2) > 0 ? t1 : t2);
}

int main()
{
. . .
```

Now the program works correctly with all data types, including the const char*, as shown in Figure 10.

Figure 10. Using the Max function template specialization for the const char data type*

Introduce another example of function template specialization. We reviewed the getValue function template for keyboard input in paragraph 1.3. A general template is acceptable

for entering numbers, but data of the 'string' type can be processed due to much simpler code. Following is the defining of a general template 'getValue' and template specialization for the 'string'.

```
#include <iostream>
#include <string>

template <class T>
void getValue(std::string prompt, T& value)
{
    std::cout << prompt;
    std::cin >> value;

    while (std::cin.fail())
    {
        std::cin.clear();
        std::cin.ignore(32767, '\n');
        std::cout << "Error... try again" << std::endl;

        std::cout << prompt;
        std::cin >> value;
    }

    std::string endLine;
    getline(std::cin, endLine);
}

template <>
inline void getValue<std::string>(std::string prompt,
    std::string& value)
{
    std::cout << prompt;
    getline(std::cin, value);
}
```

Template specialization is defined in a header file with the 'inline' keyword.

If specialization defines the inline function, its definition must be in the header file.

If specialization determines not an inline function, the header file should contain only its declaring (prototype). Defining (function body) must be in one of the implementation files.

2.4. Full Class Template Specialization

Class templates can have the same problem as function templates:

- more efficient implementation is possible for some data types than that defined in the general template;
- there is no general implementation at all for some data types.

This problem is also solved with the help of template specialization.

Full (explicit) specialization of class templates is a defining a class template where the empty angle brackets <> follow after the word template, followed by the name of the class, definite type parameters of the class, and the class defining

```
template <> className<dataTypeList>{ classDefinition }
```

An explicit class template specialization can only be defined after the generic template has already been declared (while it may not be defined). In other words, it must be known that the specialized name denotes a class template.

Let's consider an example, where the List, class — template, is a list with adding and removing elements (add,

remove) and with the functions of finding the minimum and maximum elements (getMin, getMax).

The easiest way to implement these features is to use the standard C++ tools. The most suitable mean for a dynamic list is a 'vector' — a "dynamic array" class template. To create a vector, you need to call the template constructor. The List class has a default vector constructor vector<T> list; which creates an empty dynamic data array of the T type.

The 'pushback' vector function is used in the add(T item) method for adding an element to this array — adding elements to the end of the vector.

To remove an element from the array with the specified index, the erase vector function is used in the remove(int index) method — removing an element, indicated by the pointer.

When working with vector, the pointers can be performed as iterators. Iterator is a variable of a special type points to a vector element.

To point to the first element of the vector, the following structure is usually used: vector_name.begin();. The end indication of the vector (setting after the last element) is implemented as follows: vector_name.end();.

Passing the vector elements, you can perform arithmetic operations with an iterator, as with the common pointers.

- list.begin() + index; is a pointer to the first vector element, offset to 'index' elements. That is, it is a pointer to the element with the number index.
- for (auto it = list.begin(); it < list.end(); it++) is a loop of 'it' pointer over the vector elements, starting from the first it = list.begin(); and ending the last it < list.end(); each time moving to the next element it++.

To get the element pointed to by an iterator, it can be dereferenced using the same syntax, as for the standard pointers (operator *).

*it < *pMin — the element, pointed to by the 'it' iterator (pointer), compared to the element pointed to by the 'pMin' pointer.

return *pMin; returns the value of the element pointed to by the 'pMin' pointer, i.e., the minimum value.

So, we have created a class template<class T> class List as a dynamic array, due to which you can define the maximum and minimum elements. And we are testing it with different values of the type T parameter (the program code is in the folder *Lesson05\les05_07*):

```
#include <iostream>
#include <vector>
#include <conio.h>
using namespace std;

template<class T>
class List
{
    vector<T> list;
public:
    void add(T item)
    {
        list.push_back(item);
    }

    void remove(int index)
    {
        list.erase(list.begin() + index);
    }
```

```
    auto getMin()
    {
        auto pMin = list.begin();
        for (auto it = list.begin(); it < list.end(); it++)
        {
            if (*it < *pMin)
            {
                pMin = it;
            }
        }
        return *pMin;
    }

    auto getMax()
    {
        auto pMax = list.begin();
        for (auto it = list.begin(); it < list.end(); it++)
        {
            if (*it > *pMax)
            {
                pMax = it;
            }
        }
        return *pMax;
    }
};

int main()
{
    List<int> intList;
    intList.add(4);
    intList.add(5);
    intList.add(2);
    intList.add(3);
    cout << "intList min: " << intList.getMin() << endl;
    cout << "intList max: " << intList.getMax() << endl
         << endl;
```

```
    List<string> strList;
    strList.add("four");
    strList.add("five");
    strList.add("two");
    strList.add("three");
    cout << "strList min: " << strList.getMin() << endl;
    cout << "strList max: " << strList.getMax() << endl
         << endl;

    List<const char*> txtList;
    txtList.add("four");
    txtList.add("five");
    txtList.add("two");
    txtList.add("three");
    cout << "txtList min: " << txtList.getMin() << endl;
    cout << "txtList max: " << txtList.getMax() << endl
         << endl;

    _getch();
    return 0;
}
```

Figure 11 shows the result.

```
intList min: 2
intList max: 5

strList min: five
strList max: two

txtList min: four
txtList max: three
```

Figure 11. Using the List class template

It can be seen from the result that the template works correctly with numerical ('int') and string data, but the text constants (const char*) are processed by the template incorrectly. The minimum and maximum txtList values are defined not in lexicographical order (alphabetically), as required, but just taking the first and last values from the list in the order of the pointers.

Thus, to work correctly with the const char* type, you need to develop a specialized List class template.

Let's consider the defining of a full specialization of the List class template (the program code is in the folder *Lesson05\les05_07a)*:

```
template<>
class List<const char*>
{
    vector<const char*> list;

public:
    void add(const char* item)
    {
        list.push_back(item);
    }

    void remove(int index)
    {
        list.erase(list.begin() + index);
    }

    auto getMin()
    {
        auto pMin = list.begin();
        for (auto it = list.begin(); it < list.end(); it++)
        {
            if (strcmp(*it, *pMin) < 0)
            {
```

```
                pMin = it;
            }
        }
        return *pMin;
    }

    auto getMax()
    {
        auto pMax = list.begin();
        for (auto it = list.begin(); it < list.end(); it++)
        {
            if (strcmp(*it, *pMax) > 0)
            {
                pMax = it;
            }
        }
        return *pMax;
    }
};
```

The differences between full template specializations and the general template:

- after 'template' — empty angle brackets

```
template<>;
```

- after the class name — the value of the type parameter in angle brackets

```
class List<const char*>;
```

- class template code does not use the name of the type parameter (T), only the name of data type

```
const char*
```

Figure 12 shows the result.

```
intList min: 2
intList max: 5

strList min: five
strList max: two

txtList min: five
txtList max: two
```

Figure 12. Using the full specialization of the List class template

As we see from the result, all data types have been processed correctly by now.

And you should remember that template specialization must include all the methods needed to operate on the specialization type. Imagine if we comment out the 'add' method in the template specialization, then the program *les05_07a* won't compile. And if we comment out the 'remove' method, the program will compile and run correctly because the 'remove' is not called in this program.

In the considered List class template only one-type parameter. When the class is fully specialized and has several parameters, the angle brackets will be empty after 'template', and all definite type parameters are listed after the class name in angle brackets. For example:

```
template<>
class List<const char*, int>
{. . .}
```

2.5. Partial Class Template Specialization

There are situations when it is necessary to specialize not all type parameters in a class template, but only some of them. Such a problem is solved through the partial specialization of a class template.

If a class template has multiple parameters, then you can specialize it only for one or more arguments, leaving the others unspecialized, in other words — to allow setting their values when using the template.

Partial specialization thus makes it possible to create a template that matches the general in everything except for those parameters that are replaced by actual types or values.

Partial specialization is needed when determining an more appropriate implementation for a particular set of arguments.

Let's look at an example (the program code is in the folder *Lesson05\les05_08*):

```
#include <iostream>
#include <vector>
#include <conio.h>
using namespace std;

void stopProgram(string message)
{
    cout << message << endl;
    cout << "Press any key to exit program" << endl;
    _getch();
    exit(1);
}

template <class T, int height, int width>
class Matrix
{
```

```
    T m[height][width];

public:
    const auto& operator()(int j, int i) const
    {
        if (j < 0 || j >= height
            || i < 0 || i >= width)
        {
            stopProgram("Matrix index error!");
        }
        return m[j][i];
    }

    auto& operator()(int j, int i)
    {
        if (j < 0 || j >= height
            || i < 0 || i >= width)
        {
            stopProgram("Matrix index error!");
        }
        return m[j][i];
    }

    auto getMin()
    {
        auto min = m[0][0];
        for (int j = 0; j < height; j++)
        {
            for (int i = 0; i < width; i++)
            {
                if (m[j][i] < min)
                {
                    min = m[j][i];
                }
            }
        }
        return min;
    }
```

```
    auto getMax()
    {
        auto max = m[0][0];
        for (int j = 0; j < height; j++)
        {
            for (int i = 0; i < width; i++)
            {
                if (m[j][i] > max)
                {
                    max = m[j][i];
                }
            }
        }
        return max;
    }
};

int main()
{
    const int height = 2;
    const int width = 3;

    Matrix<int, height, width> intMatrix;

    intMatrix(0, 0) = 5;
    intMatrix(0, 1) = 1;
    intMatrix(0, 2) = 9;
    intMatrix(1, 0) = 15;
    intMatrix(1, 1) = 11;
    intMatrix(1, 2) = 19;

    cout << "intMatrix min: " << intMatrix.getMin()
         << endl;
    cout << "intMatrix max: " << intMatrix.getMax()
         << endl << endl;

    Matrix<string, height, width> strMatrix;
```

```
    strMatrix(0, 0) = "five";
    strMatrix(0, 1) = "one";
    strMatrix(0, 2) = "nine";
    strMatrix(1, 0) = "fifteen";
    strMatrix(1, 1) = "eleven";
    strMatrix(1, 2) = "nineteen";
    cout << "strMatrix min: " << strMatrix.getMin()
         << endl;
    cout << "strMatrix max: " << strMatrix.getMax()
         << endl << endl;

    Matrix<const char*, height, width> txtMatrix;
    txtMatrix(0, 0) = "five";
    txtMatrix(0, 1) = "one";
    txtMatrix(0, 2) = "nine";
    txtMatrix(1, 0) = "fifteen";
    txtMatrix(1, 1) = "eleven";
    txtMatrix(1, 2) = "nineteen";

    cout << "txtMatrix min: " << txtMatrix.getMin()
         << endl;
    cout << "txtMatrix max: " << txtMatrix.getMax()
         << endl << endl;

    _getch();
    return 0;
}
```

'Matrix' class-template represents static two-dimensional array (matrix) with an option to modify the values of elements (via overloaded operator ()) and with the functions for finding the minimum and maximum elements (getMin, getMax).

The matrix dimensions are set with the non-type height, width class parameters. The 'main' function tests the Matrix with different data types.

Figure 13 shows the result.

```
intMatrix min: 1
intMatrix max: 19

strMatrix min: eleven
strMatrix max: one

txtMatrix min: five
txtMatrix max: nineteen
```

Figure 13. Using the Matrix class template

It can be seen from the result that the template works correctly with numerical (int) and string data, but the text constants (const char*) are processed by the template incorrectly. The minimum and maximum values of the txtMatrix are defined not in lexicographic order (alphabetically), as required, but just taking the first and last values from the two-dimensional array in the order of the pointers.

Thus, to work correctly with the const char* type, you need to develop a specialized Matrix class template.

But here comes a little problem. The matrix template parameters can also be its dimensions. It turns out that for each new combination of sizes, you will have to create a separate class template specialization, like this:

```
template <>
class Matrix<const char*, 2, 3>

template <>
class Matrix<const char*, 3, 2>
```

and so on.

The solution of the problem is to create a partial specialization where only the first template parameter specializes (T type parameter).

Let's look at how defines a partial specialization of Matrix class template (the program code is in the folder *Lesson05\les05_08a*):

```
template <int height, int width>
class Matrix<const char*, height, width>
{
    const char* m[height][width];

public:
    const auto& operator()(int j, int i) const
    {
        if (j < 0 || j >= height
            || i < 0 || i >= width)
        {
            stopProgram("Matrix index error!");
        }
        return m[j][i];
    }

    auto& operator()(int j, int i)
    {
        if (j < 0 || j >= height
            || i < 0 || i >= width)
        {
            stopProgram("Matrix index error!");
        }
        return m[j][i];
    }

    auto getMin()
    {
        auto min = m[0][0];
```

```
        for (int j = 0; j < height; j++)
        {
            for (int i = 0; i < width; i++)
            {
                if (strcmp(m[j][i], min) < 0)
                {
                    min = m[j][i];
                }
            }
        }
        return min;
    }

    auto getMax()
    {
        auto max = m[0][0];
        for (int j = 0; j < height; j++)
        {
            for (int i = 0; i < width; i++)
            {
                if (strcmp(m[j][i], max) > 0)
                {
                    max = m[j][i];
                }
            }
        }
        return max;
    }
};
```

The differences between partial and full template specializations:

- after 'template' — non-specialized parameters are set into the angle brackets:

```
template <int height, int width>;
```

- after the class name — the values of the specialized parameters are set into the angle brackets (const char*), as well as the names of non-specialized parameters (height, width):

```
class Matrix<const char*, height, width>
```

Figure 14 shows the result.

```
intMatrix min: 1
intMatrix max: 19

strMatrix min: eleven
strMatrix max: one

txtMatrix min: eleven
txtMatrix max: one
```

Figure 14. Using the partial specialization of the Matrix class template

As we see from the result, all data types have been processed correctly by now.

2.6. Examples of Creating Class Templates

Consider an example of a simple template of a 'Pair' class.

We quite often have to deal with structures consisting of two elements, such as:

- Point(int x, int y) — the coordinates of the points;
- Point(double x, double y) — the points coordinates on the coordinate plane;

- Fraction(int x, int y) — fraction;
- Money(int dollar, int cent) — a sum of money;
- Name(string firstName, string surname) — name and surname;
- Mark(Name student, int mark) — student's mark.

It would probably be convenient to have a class/structure template that could define basic operations with all possible structures of this type.

Let's analyze the using of the Pair class template in the following example (the program code is in the folder *Lesson05\les05_09*):

```
#include <iostream>
#include <conio.h>
using namespace std;

template <typename T1, typename T2>
struct Pair
{
    T1 first;
    T2 second;
    Pair() : first(T1()), second(T2()) {};
    Pair(const T1& x, const T2& y) : first{x}, second{y}{}

    void display()
    {
        cout << "(" << first << "," << second << ")";
    }

    bool operator==(const Pair& pair)
    {
        return this->first == pair.first &&
            this->second == pair.second;
    }
```

```
    bool operator!=(const Pair& pair)
    {
        return !(*this == pair);
    }
};

int main()
{
    Pair<int, int>point1(0, 0);
    Pair<int, int>point2(1, 1);
    point1.display();
    cout << " ";
    point2.display();
    cout << " ";
    cout << (point1 == point2 ? "equals" : "not equals")
         << endl;

    Pair<int, int> point;
    point.display();
    cout << " ";
    point1.display();
    cout << " ";
    cout << (point != point1 ? "not equals" : "equals")
         << endl;

    Pair<string, string> name("John", "Smith");
    Pair <Pair<string, string>, int > mark(name, 8);
    mark.first.display();
    cout << " : " << mark.second << endl;
    _getch();
    return 0;
}
```

The 'Pair' structure template has two type parameters: T1 and T2. Obviously, they define the types of the first and the second elements of the pair.

The 'Pair()' default constructor inputs the default values of T1 and T2 types into the first and the second elements of the pair.

Constructor with parameters Pair(const T1& x, const T2& y) keeps values of the parameters.

The display() function outputs the values of the structure.

Overridden operators == and != compare the values of the structures.

The 'main' function tests the class template for different data types (see Figure 15).

```
(0,0) (1,1) not equals
(0,0) (0,0) equals
(John,Smith) : 8
```

Figure 15. Using the Pair class template

To create a pair mark(name, 8), which has a pair of the first, it requires a nested specification of type parameters:

```
Pair<string, string> name("John", "Smith");
Pair <Pair<string, string>, int > mark(name, 8);
```

We cannot use the display() function for the 'mark' variable, because the output operator << is not defined for the mark.first (with the <string, string> type). But we can display the data of this structure in this way:

```
    mark.first.display();
    cout << " : " << mark.second << endl;
```

Analyze another example with the "Matrix (two-dimensional array)" class template (the program code is in the folder *Lesson05\les05_10*).

You can find the class template code in the file *Matrix.h*:

```
#pragma once
#include <iostream>

void stopProgram(std::string message);

template <class T>
class Matrix
{
protected:
    int height;
    int width;
    T** m;

private:
    bool allocate(int height, int width)
    {
        if (height <= 0 && width <= 0)
        {
            return false;
        }

        this->height = height;
        this->width = width;
        m = new T * [height];
        for (int j = 0; j < height; j++)
        {
            m[j] = new T[width];
            for (int i = 0; i < width; i++)
            {
                m[j][i] = T();
            }
        }
```

```
        return true;
    }

    void clear()
    {
        if (m != nullptr)
        {
            for (int j = 0; j < height; j++)
            {
                delete[] m[j];
            }
            delete[] m;
        }
        m = nullptr;
    }

    void copyTo(T** from)
    {
        for (int j = 0; j < height; j++)
        {
            for (int i = 0; i < width; i++)
            {
                m[j][i] = from[j][i];
            }
        }
    }

public:
    Matrix(int height, int width)
    {
        if (height <= 0 || width <= 0)
        {
            stopProgram("Wrong sizes!");
        }

        if (!allocate(height, width))
        {
```

```
            stopProgram("Wrong sizes!");
        }
    }

    Matrix(int size) : Matrix(size, size)
    {}
    Matrix()
    {
        height = width = 0;
        m = nullptr;
    }
    Matrix(const Matrix& matrix)
    {
        clear();
        allocate(matrix.height, matrix.width);
        copyTo(matrix.m);
    }
    ~Matrix()
    {
        clear();
    }

    auto& operator()(int j, int i)
    {
        if (j < 0 || j >= height
            || i < 0 || i >= width)
        {
            stopProgram("Matrix index error!");
        }
        return m[j][i];
    }

    Matrix<T>& operator=(const Matrix<T>& matrix)
    {
        if (&matrix == this)
        {
            return *this;
        }
```

```
        clear();
        allocate(matrix.height, matrix.width);
        copyTo(matrix.m);

        return *this;
    }

    friend std::ostream& operator<<(std::ostream & output,
                                    const Matrix& matrix)
    {
        if (matrix.m == nullptr)
        {
            output << "Empty matrix" << std::endl;
        }

        for (int j = 0; j < matrix.height; j++)
        {

            for (int i = 0; i < matrix.width; i++)
            {
                output << " " << matrix.m[j][i];
            }
            output << std::endl;
        }

        return output;
    }
};
```

stopProgram is a function prototype, which crashes the program because of errors. Function body is in the file *les05_10.cpp*.

The class objects will store a two-dimensional dynamic array of elements of the T type. That's why the class fields are:

- reference to the two-dimensional dynamic array T** m;

- array dimensions int height, int width (height and width of the matrix).

Operations on allocating dynamic memory, freeing memory, copying elements are constantly used in constructors, destructors, and the 'Matrix' class operator overloading. So, it makes sense to write and apply the following internal, private functions:

- bool allocate(int height, int width) — allocating memory for matrix;
- void clear() — deallocating matrix memory;
- void copyTo(T** from) — copying matrix elements.

Class constructors:

- Matrix(int height, int width) — for creating a matrix with 'height' and 'width' dimensions. Uses the 'allocate' function to allocate memory;
- Matrix(int size) — for creating a square matrix with 'size';
- Matrix() — for creating an empty matrix;
- Matrix(const Matrix& matrix) — a copy constructor to copy matrix correctly. Uses the 'clear', 'allocate', and 'copyTo' functions for freeing and allocating memory and copying matrix elements.

Class destructor: ~Matrix() — uses 'clear' function.

The overloaded operator `()` is used to access matrix elements: auto& operator()(int j, int i) — for reading and writing matrix elements.

For operations with matrix assigning, we use the overloaded operator =: Matrix<T>& operator=(const Matrix<T>& matrix) — uses the functions: 'clear', 'allocate', 'copyTo'.

The overloaded operator << is used to display the matrix on the screen.

```
friend std::ostream& operator<<(std::ostream & output,
                                const Matrix& matrix)
```

The Matrix class is used for matrices, which elements are the data of any type, for instance:

```
Matrix<int> intMatrix(3);

Matrix<string> strMatrix(2, 3);

Matrix<Point> pointMatrix(2, 3);
```

The operation kit is minimized: creating a matrix, filling with data, changing data, accessing individual values, displaying on the screen.

Other general purpose operations can be added optionally: finding the maximum and minimum values, various sorts, and searches.

But what should we do, if we want to enter numbers and perform operations on numerical matrices like addition, subtraction, multiplication?

You should not add such operations to the Matrix class template because they will make no sense to all the non-numeric type T parameters. You can use a template specialization. But the simplest solution seems will be using inheritance.

Inheritance is one of the fundamental principles of OOP. Based on it, the class can use the variables and class methods as its own from which it is inherited.

Inheritance is implemented like this:

```
class derived_class : public base_class
{
. . .
};
```

where derived_class — the inherited class. It is called: derived class, child class, subclass;

base_class — the name of the class from which the inheritance is implemented. This class is called: base class, parent class, superclass.

The child class can use any base class variables and functions that the 'public' or 'protected' access modifiers have.

It is obvious that the child class defines its variables and functions. This class is created just for this.

You can find a code with the NumberMatrix class template in the file *NumberMatrix.h:*

```
#pragma once
#include "matrix.h"

template <class T>
class NumberMatrix : public Matrix<T>
{

public:
    NumberMatrix(int height, int width) :
          Matrix<T>(height, width)
    {}

    NumberMatrix(int size) : NumberMatrix(size)
    {}
```

```
    void setRand(int max = 100, int min = 0)
    {
        for (int j = 0; j < this->height; j++)
        {
            for (int i = 0; i < this->width; i++)
            {
                this->m[j][i] = (T)(min + rand() %
                                    (max - min + 1));
            }
        }
    }

    friend NumberMatrix<T> operator+
           (const NumberMatrix<T>& m1,
        const NumberMatrix<T>& m2)
    {
        if (m1.height != m2.height ||
            m1.width != m2.width)
        {
            stopProgram("Matrix sizes do not match!");
        }

        NumberMatrix<T> mRes(m1.height, m1.width);
        for (int j = 0; j < mRes.height; j++)
        {
            for (int i = 0; i < mRes.width; i++)
            {
                mRes.m[j][i] = m1.m[j][i] + m2.m[j][i];
            }
        }
        return mRes;
    }

    friend NumberMatrix<T> operator-
           (const NumberMatrix<T>& m1,
        const NumberMatrix<T>& m2)
    {
```

```
        if (m1.height != m2.height || m1.width != m2.width)
        {
            if (m1.height != m2.height ||
                m1.width != m2.width)
            {
                stopProgram("Matrix sizes do not match!");
            }
        }

        NumberMatrix<T> mRes(m1.height, m1.width);
        for (int j = 0; j < mRes.height; j++)
        {
            for (int i = 0; i < mRes.width; i++)
            {
                mRes.m[j][i] = m1.m[j][i] - m2.m[j][i];
            }
        }
        return mRes;
    }

    friend std::ostream& operator<<(std::ostream& output,
        const NumberMatrix& matrix)
    {
        if (matrix.m == nullptr)
        {
            output << "Empty matrix" << std::endl;
        }
        for (int j = 0; j < matrix.height; j++)
        {
            for (int i = 0; i < matrix.width; i++)
            {
                output << " " << matrix.m[j][i];
            }
            output << std::endl;
        }
        return output;
    }
};
```

The 'NumberMatrix' class template is inherited from the 'Matrix' class. It is written next way:

```
template <class T>
class NumberMatrix : public Matrix<T>
```

the type <T> parameter is indicated in angle brackets after the name of the base class.

The 'NumberMatrix' class fields are protected member variables of the 'Matrix' base class.

```
protected:
    int height;
    int width;
    T** m;
```

The 'NumberMatrix' constructors call the corresponding base class constructors:

```
NumberMatrix(int height, int width) : Matrix<T>(height,
                                                width)
NumberMatrix(int size) : NumberMatrix(size)
```

The class has a setRand(int max = 100, int min = 0) method, which fills the matrix with random values.

Math operations on numeric matrices allow you to execute overloaded operators + and –:

```
friend NumberMatrix<T> operator+(const NumberMatrix<T>& m1,
                                 const NumberMatrix<T>& m2)

friend NumberMatrix<T> operator-(const NumberMatrix<T>& m1,
                                 const NumberMatrix<T>& m2)
```

The NumberMatrix objects are displayed due to the overloaded operator <<

```
friend std::ostream& operator<<(std::ostream& output,
                                const NumberMatrix& matrix)
```

The Matrix and NumberMatrix class templates are tested in the main function (*file les05_10.cpp*).

```
#include <conio.h>
#include "numbermatrix.h"

using namespace std;

struct Point
{
    int x;
    int y;

    Point(int x = 0, int y = 0) : x{x}, y{y}
    {}

    friend bool operator>(const Point& p1, const Point& p2)
    {
        double d1 = sqrt(p1.x * p1.x + p1.y * p1.y);
        double d2 = sqrt(p2.x * p2.x + p2.y * p2.y);
        return d1 - d2;
    }

    friend std::ostream& operator<<(std::ostream& output,
                                    const Point& p)
    {
        output << "(" << p.x << "," << p.y << ")";
        return output;
    }
};
```

```
void stopProgram(string message)
{
    cout << message << endl;
    cout << "Press any key to exit program" << endl;
    _getch();
    exit(1);
}

int main()
{
    cout << "Class Matrix" << endl << endl;

    Matrix<int> intMatrix(3);
    for (int j = 0; j < 3; j++)
    {
        for (int i = 0; i < 3; i++)
        {
            intMatrix(j, i) = j * 10 + i;
        }
    }
    cout << "int Matrix:" << endl;
    cout << intMatrix << endl;

    Matrix<int> intMatrixA = intMatrix;
    cout << "Copy int Matrix:" << endl;
    cout << intMatrixA << endl;

    Matrix<string> strMatrix(2, 3);
    strMatrix(0, 0) = "string00";
    strMatrix(0, 1) = "string01";
    strMatrix(0, 2) = "string02";
    strMatrix(1, 0) = "string10";
    strMatrix(1, 1) = "string11";
    strMatrix(1, 2) = "string12";
    cout << "string Matrix:" << endl;
    cout << strMatrix << endl;
```

```
    Matrix<Point> pointMatrix(2, 3);
    pointMatrix(0, 0) = Point(0, 0);
    pointMatrix(0, 1) = Point(0, 1);
    pointMatrix(0, 2) = Point(0, 2);
    pointMatrix(1, 0) = Point(1, 0);
    pointMatrix(1, 1) = Point(1, 1);
    pointMatrix(1, 2) = Point(1, 2);
    cout << "point Matrix:" << endl;
    cout << pointMatrix << endl;

    NumberMatrix<double> numberMatrixA(2, 3);
    numberMatrixA.setRand();
    cout << "double Number MatrixA:" << endl;
    cout << numberMatrixA << endl;

    NumberMatrix<double> numberMatrixB(2, 3);
    numberMatrixB.setRand();
    cout << "double Number MatrixB:" << endl;
    cout << numberMatrixB << endl;

    NumberMatrix<double> numberMatrixC = numberMatrixA +
                                         numberMatrixB;
    cout << "MatrixC = MatrixA + MatrixB:" << endl;
    cout << numberMatrixC << endl;
    _getch();
    return 0;
}
```

The 'Point' structure is used to test the functionality of the 'Matrix' template with a user (standalone — not built-in) data types.

The 'stopProgram' function is called in the 'Matrix' class to crash the program in case of data errors.

In the 'main' function, the type matrices are created as a 'Matrix' type from the 'int', 'string', and 'Point' type elements;

access functions are checked to the elements, copying, and data output.

Matrices of the NumberMatrix type are also created, and the matrix addition function is checked.

The program result is shown in Figure 16.

```
Class Matrix

int Matrix:
 0 1 2
 10 11 12
 20 21 22

Copy int Matrix:
 0 1 2
 10 11 12
 20 21 22

string Matrix:
 string00 string01 string02
 string10 string11 string12

point Matrix:
 (0,0) (0,1) (0,2)
 (1,0) (1,1) (1,2)

double Number MatrixA:
 41 85 72
 38 80 69

double Number MatrixB:
 65 68 96
 22 49 67

MatrixC = MatrixA + MatrixB:
 106 153 168
 60 129 136
```

Figure 16. Using the Matrix and NumberMatrix class templates

The above example shows that different, at first sight, unrelated C++ features (class templates, operator overloading, inheritance) can (and should!) be successfully combined to write high-quality programs.

3. Variadic Templates

We know that there are functions in C++ with a variable number of parameters (variadic). Such functions are declared in the same way as ordinary functions, but instead of the last ellipsis is put in the first parameter:

```
return_type function_name(parameter_list, ...)
```

Example (the program code is in the folder *Lesson05\les05_11*):

```
#include <iostream>
#include <conio.h>
using namespace std;

int getSum(int count, int first, ...)
{
    int sum = 0;
    int* p = &first;
    while (count--)
    {
        sum += *p;
        p++;
        p++;
    }
    return sum;
}

int main()
{
    int sum = getSum(5, 1, 2, 3, 4, 5);
    cout << "sum = " << sum << endl;
    sum = getSum(4, 25, 0, 5, 60);
```

```
    cout << "sum = " << sum << endl;

    _getch();
    return 0;
}
```

The first function parameter of the 'getSum' is the number of parameters in the variable part. The function selects and sums the values of all these parameters, knowing that they are located in memory sequentially after the 'first' parameter, and all these parameters are integers.

The program result is shown in Figure 17.

```
sum = 15
sum = 90
```

Figure 17. Using the variadic functions

As of C++11, C++ has let to define templates that take a variable number of parameters.

A template with a variable number of parameters (Variadic Template) is a template with an unknown number of parameters. Variadic template syntax:

```
template<class... Types>
```

where 'Types' is a type pack. A type pack can contain type parameters and non-type parameters.

A type pack can be used in a function's parameter list:

```
template<class... Types>
void func(Types... parameters)
```

where 'parameters' is like a pack of parameters whose types are specified by the specified 'Types' type packs, a parameter pack can only be in a template function, and it must match the type pack.

You can use both a type and parameter pack inside a function. To do this, we put an ellipsis after the pack name . . . , which is interpreted by the compiler as expanding the pack into a comma-separated list at the specified position. The ordinary way of defining the variadic template functions is a recursive defining.

Consider an example (the program code is in the folder *Lesson05\les05_12*):

```
#include <iostream>
#include <conio.h>
using namespace std;

double getSum(double x)
{
    return x;
}

template<class... Args>
double getSum(double x, Args... args)
{
    return x + getSum(args...);
}

int main()
{
    double sum = getSum(1, 2, 3, 4, 5);
    cout << "sum = " << sum << endl;

    sum = getSum(25, 0, 5, 60);
```

```
    cout << "sum = " << sum << endl;

    _getch();
    return 0;
}
```

How does it work?

The call of the getSum(25, 0, 5, 60) function at the compiling step results in creating and calling of a getSum(25, ...) function, where will be an operator inside it

```
return 25 + getSum(0, ...);
```

which in turn, will result in the creating and calling of the getSum(0, ...) function, and so on…

Recursive descent (recursive call of the getSum with a shortening list of parameters) will be stopped when only 1 parameter remains and a regular double getSum(double x) function that left is called.

While the program is running, all these functions will sequentially call each other and, thus, they sum the parameters getSum(25, 0, 5, 60).

The program result is shown in Figure 18.

```
sum = 15
sum = 90
```

Figure 18. Using a variadic function template

Another example we are going to consider is the analogue of the Print function (the program code is in the folder *Lesson05\les05_13*):

```
#include <iostream>
#include <conio.h>
using namespace std;

void print()    // implements nothing
{}

template <class First, class... Other>
void print(const First& first, const Other&... other)
{
    cout << first;
    print(other...);
}

template <class... Args>
void println(const Args&... args)
{
    print(args...);
    cout << endl;
}

struct Point
{
    int x;
    int y;

    friend std::ostream& operator<<(std::ostream& output,
                                    const Point& p)
    {
        output << "(" << p.x << "," << p.y << ")";
        return output;
    }
};

int main()
{
    println("Hello, world");
```

```
    println("Pi = ", 3.14, '\n', 2, " * ", 2, " = ", 2 * 2);

    Point point{ 12,2 };
    println("Point: ", point);

    _getch();
    return 0;
}
```

The basic idea of this program is the same as in the previous example: recursion creates a chain of functions, and each function displays a parameter on the screen.

The main function is void print(const First& first, const Other&... other).

The 'println' adds passing to a new line to the 'print'.

The void print() function is needed to complete the recursion.

Parameter types can be varied, if only they have an overloaded output operator <<.

In the example above, the 'println' function prints integers and floating numbers, characters, arrays of characters, values of the 'Point' structure.

The program result is shown in Figure 19.

```
Hello, world
Pi = 3.14
2 * 2 = 4
Point: (12,2)
```

Figure 19. Using variadic 'print' function template

4. Applying std::initializer_list

For static and dynamic arrays initialization it is convenient to use an initialization list:

```
int array[] { 7, 6, 5, 4, 3, 2, 1 };

int* dynArray = new int[7] { 7, 6, 5, 4, 3, 2, 1 };
```

We can create a class with conditions analogous to a dynamic array (the program code is in the folder *Lesson05\les05_14*):

```
class IntArray
{

private:
    int length;
    int* data;

public:
    IntArray() : length(0), data(nullptr)
    {
    }

    IntArray(int length) : length(length)
    {
        data = new int[length];
    }

    ~IntArray()
    {
        delete[] data;
    }
```

```
    int& operator[](int index)
    {
        return data[index];
    }

    int getLength() const
    {
        return length;
    }
};
```

But, we cannot initialize an object of this class as an array.

```
int main()
{
    IntArray array{ 7, 6, 5, 4, 3, 2, 1 }; // compilation
error
    for (int i = 0; i < 7; i++)
    {
        cout << array[i] << ' ';
    }
    cout << endl;

    _getch();
    return 0;
}
```

IntArray array{ 7, 6, 5, 4, 3, 2, 1 }; string causes a compilation error.

```
Error C2440  'initializing': cannot convert from
             'initializer list' to 'IntArray'
```

C++ compiler (as of C++11) automatically converts initialization list into an object of std::initializer_list type.

Therefore, for that a class object could be initialized with a list; it will be enough to create a constructor in this class. Such a constructor will take an object of the std::initializer_list type as a parameter.

For working on the std::initializer_list, you must assign the header file <initializer_list>. The std::initializer_list class is a class template. That's why we must indicate used data type in angle brackets after the std::initializer_list. For e.g.: std::initializer_list<int> or std::initializer_list<std::string>. To determine the number of elements in a list, the std::initializer_list has the size() function.

Let's add the IntArray constructor to our class that gets the std::initializer_list data type (the program code is in the folder *Lesson05\les05_14a*):

```
IntArray(const std::initializer_list<int>& list) :
         IntArray(list.size())
{
    int i = 0;

    for (auto& element : list)
    {
        data[i] = element;
        i++;
    }
}
```

The initialization constructor works in the following way:

- firstly, we create the *data array of the list.size() size by calling the IntArray(list.size()), constructor;
- then, we fill this array in a loop with the elements from the initializer_list<int>list parameter list list.

The program result is shown in Figure 20.

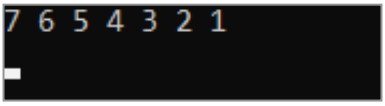


Figure 20. Using the std::initializer_list type to create a constructor with an initialization list

Likewise, you can add to a class template an option to use an initialization list.

Let's transform the IntArray class to the DynArray class template (the program code is in the folder *Lesson05\les05_14b*):

```
#include <iostream>
#include <initializer_list>
#include <conio.h>

using namespace std;

template <class T>
class DynArray
{
private:
    int length;
    T* data;

public:
    DynArray() : length(0), data(nullptr)
    {
    }

    DynArray(int length) : length(length)
    {
        data = new T[length];
    }
```

```
    DynArray(const std::initializer_list<T>& list) :
             DynArray(list.size())
    {
        int i = 0;
        for (auto& element : list)
        {
            data[i] = element;
            i++;
        }
    }

    ~DynArray()
    {
        delete[] data;
    }
    T& operator[](int index)
    {
        return data[index];
    }
    int getLength() const
    {
        return length;
    }
};

struct Point
{
    int x;
    int y;

    friend std::ostream& operator<<(std::ostream& output,
                                    const Point& p)
    {
        output << "(" << p.x << "," << p.y << ")";
        return output;
    }
};
```

```
int main()
{
    DynArray<int> intArray{ 7, 6, 5, 4, 3, 2, 1 };
    for (int i = 0; i < 7; i++)
    {
        cout << intArray[i] << ' ';
    }
    cout << endl;

    DynArray<Point> pointArray{Point{1,1}, Point{7, 7},
                               Point{3,3} };
    for (int i = 0; i < 3; i++)
    {
        cout << pointArray[i] << ' ';
    }
    cout << endl;
    _getch();
    return 0;
}
```

By now, an initialization constructor transmits a type T parameter of the DynArray class template as a type parameter of the initializer_list class template.

```
DynArray(const std::initializer_list<T>& list) :
         DynArray(list.size())
```

Other items will stay without any modifications. The program result is shown in Figure 21.

```
7 6 5 4 3 2 1
(1,1) (7,7) (3,3)
```

Figure 21. Using the std::initializer_list type to create a constructor with an initialization list in a class template

The initialization list is used for an array of integers

```
DynArray<int> intArray{ 7, 6, 5, 4, 3, 2, 1 };
```

and for an array of coordinates

```
DynArray<Point> pointArray{ Point{1,1}, Point{7, 7},
                            Point{3,3} }
```

5. The ‘string’ Class

5.1. Characters and Code Chart Representation

Computers are designed for computing. RAM stores numbers. The processor processes the numbers. Text is also stored as a set of numbers. Every letter corresponds to a numeric code. Programs process these codes and output them in graphic symbols on the screen or on paper, if necessary. Codes are set into tables, which are called character encodings. Encodings are different.

Different in number of encoded characters, in the code length, in correspondence of codes to characters. It is historically formed.

The first and the simplest form standard character encodings are the ASCII chart (American standard code for information interchange).

This chart includes all user needs: control characters, the Latin alphabet characters, numbers, and punctuation symbols (see Figure 22).

ASCII Code Chart

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Figure 22. ASCII Chart

This is a character encoding with codes from 0 to 127, in binary form — from 00000000 to 11111111, in hexadecimal — from 00 to 7F. 7 bits are required for storing the values of this code.

ASCII does not support national alphabets, and it has not enough space for that. So, various national 8-bit character encodings were created (codes from 0 to 255), in which the first 127 characters match with ASCII. And the second part of the chart (from 128 to 255) included symbols of other alphabets, e.g., Windows-1250 -character encoding of Central European languages, Windows-1251 — character encoding of languages with Cyrillic, Windows-1253 — Greek character encoding, etc.

Each character occupies precisely 1 byte of computer memory when using such encodings. And it is convenient. But it is very inconvenient to work on texts that require several encodings at the same time. And later, to solve this problem, one big general chart with codes that included all characters from all existing alphabets in the world was created. This chart is the Unicode.

Notice that Unicode is not a character encoding. Encoding characterizes:

- correspondence of codes and symbols;
- method of storing codes in computer memory.

The encoding is implemented in the software that works with corresponded encoding.

Unicode is a code chart, a universal character set. It consists of 1,114,112 items grouped into 17 blocks of 65536 symbols. The zero block is the main one; it contains the symbols

of all modern alphabets. Most of the blocks have not been filled yet.

With such a list of codes, you can come up with several different ways of placing codes in memory; besides, each will be effective. These techniques are called a Unicode Transformation Format (UTF). Unicode has the following representations:

- UTF-8;
- UTF-16;
- UTF-32.

UTF-8 encoding is compatible with 8-bit encoding. The first 128 characters of UTF-8 are encoded using a single byte with the same binary value as ASCII. So that valid ASCII text (e.g., program text in C++) is valid UTF-8-encoded Unicode as well.

Such an advantage turns into certain disadvantages in other aspects of use. The encoded Unicode characters greater than 128 of UTF-8 are encoded using from 2 to 6 bytes. In other words, UTF-8-encoded Unicode is a variable-length encoding. Frequently used characters (ASCII and national scripts) occupy from 1 to 2 bytes; rarely used characters have a large length. In this way, UTF-8 is conveniently used for storing texts on external storage to save space, but on the other hand, it is less convenient for processing texts in RAM due to different lengths of characters in bytes.

UTF-16 is also a variable-length encoding. The main difference from UTF-8 is it has two bytes as a basic unit, not a single byte. That is, in UTF-16 encoding, any Unicode character can be encoded with either two or four bytes. In

other words, ASCII characters and national alphabets (zero block in Unicode) are recorded in 2 bytes, rarer characters — in 4 bytes.

Practically, UTF-16 is much more convenient for RAM processing — all characters are 2 bytes. But it takes an extra space for ASCII-character.

UTF-32 — encoded Unicode used to encode Unicode code points that use exactly 32 bits per code point. UTF-32 is a direct, fixed-length encoding.

The main advantage of UTF-32 is that the Unicode code points are directly indexed. Finding the Nth code point in a sequence of code points is a constant operation. In contrast, a variable-length code requires sequential access to find the Nth code point in a sequence. It makes UTF-32 a simple replacement in code that uses integers that are incremented by one to examine each location in a string, as was commonly done for ASCII.

The main disadvantage of UTF-32 is that it is space-inefficient, using four bytes per code point. Characters beyond the Basic Multilingual Plane (BMP) are rare in most texts. It makes UTF-32 close to twice the size of UTF-16.

Thus, we see that in programming languages:

- it is convenient to use the ASCII or UTF-8 character encoding for processing texts containing only Latin script;
- it is convenient to use UTF-16 for text processing and UTF-8 for saving on external storage texts containing symbols of national alphabets;
- it makes sense to use UTF-32 for processing texts with non-standard characters.

5.2. Character Data Type and Strings in C++

C++ has the following (built-in) data types for storing character code:

```
char     ch{ 'a' };
char8_t  ch8{ u8'a' }; // C++ is required 20
char16_t ch16{ u'a' };
char32_t ch32{ U'a' };
wchar_t  chw{ L'a' };  // a non-standard the Microsoft
                       // compiler type
```

The ‘char’ type is the source type for storing characters in C and C++. This type can be used to store characters in ASCII encoding or in any 8-bit (one-byte) encoding and store individual bytes of multibit Unicode characters.

The ‘char’ has a capacity of 8 bits, and it is represented in both variants of modifications: signed char and unsigned char. The ‘char’ means signed char. That is, such type variables (char ch) can have values from -128 to 127.

The ‘wchar_t’ type is an implementation-defined wide character type. It is shown as a 16-bit character in the Microsoft compiler used to store UTF-16 encoded Unicode.

The ‘char8_t’, ‘char16_t’, and ‘char32_t’ types are as the 8-bit, 16-bit, and 32-bit characters (the ‘char8_t’ appeared in C++20). UTF-8-encoded Unicode characters are stored in the ‘char8_t’ type. UTF-16-encoded Unicode characters are stored in the ‘char16_t’ type. And UTF-32-encoded Unicode characters are stored in the ‘char32_t’ type.

C++ contains different types of strings to store data represented by different types of characters:

- string with the ‘char’ type parameters;

- ‘wstring’ with the ‘wchar_t’ type parameters;
- ‘u16string’ with the ‘char16_t’ type parameters;
- ‘u32string’ with the ‘char32_t’ type parameters.

All of these types are specializations of the ‘basic_string’ class template based on relevant character types. The ‘string’ type, for e.g., is defined as follows:

```
typedef basic_string<char> string;
```

Thus, all necessary functions are set in the ‘basic_string’ class template, to work with strings of different types. Let’s consider them through the example of the ‘string’ class.

5.3. Purpose and Objectives of the ‘string’ Class

We know that to process text data in C++ we can use a character array where the last element has a value ‘\0’ (null character).

```
char text[20]{ "This is a C-string" };
```

Such character arrays are often called C-strings. Character arrays were called in such a way in C++ because they are a heritage in C language, where they were the only type for storing texts.

And in C ++ a new data type for processing text information appeared.

```
string text{ "This is a C++ string" };
```

These are also strings, but C++ strings. Thus, there are two types of strings in C++: C-strings and C++ strings.

Working on C-strings is efficient in terms of speed and time, but it has significant functional drawbacks:

- they cannot be compared using comparison operators (==, <, >);
- you cannot assign the values of one array to another;
- although there is a large set of special functions for working with C strings, but not all of these functions are intuitive and easy to use.

The significant disadvantage is the impossibility of dynamic resizing and the odds of being out of the array when working on it. The ‘string’ class was created and included in the standard library to solve these problems.

Benefits of the ‘string’ class compared to C strings:

- the ‘string’ class object is a dynamic array, so it is easy to modify;
- working on this object is protected from addressing errors;
- for working with the ‘string’ objects, the class has a large set of suitable functions;
- the ‘string’ class is a container. In C++ language, a container as a class is created to store and organize multiple objects of a particular data type. So, you can use the full set of functions and operations common to all containers for working on the ‘string’ objects.

5.4. The ‘string’ Class Analysis

5.4.1. Declaring, initializing, and accessing elements

To work with objects of the ‘string’ class (‘strings’ further), it will be enough for you to connect headers.

```
#include <string>
```

As of C++ 17, the <string> header file is included in the <iostream> header file. Thus, in console programs for working with strings, it is enough to write:

```
#include <iostream>
```

The 'string' is in the std namespace. So, to declare a variable (without "using namespace std;") you need to write:

```
std::string aString;
```

Declaring a string without its initializing creates an empty string (with the 'aString' value, after executing the above string will be "").

If we write like this:

```
string text;
int number;
cout << "|" << number << "|" << text << "|" << endl;
```

then the compiler will output an error message only for the 'number' variable:

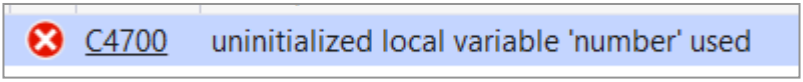


Figure 23

The integer variable int number does not have any value. However, the string variable string text has an empty value.

You can initialize strings in several ways:

- with a constant array of characters:

```
string text{ "Hello" }; // const char* string
```

- with repetition of characters (in brackets!)

```
string name( 8, 'a' );  // string of 8 'a' characters
```

- with using the assignment operator

```
string month = "March"; // the same as the string
                        // month{"March"};
```

As opposed to C strings, the 'string' objects do not contain the end null character '\0'. '\0' character can be in a string as a regular character, for example:

```
char stringC[]{ 'a', 'b', 'c', '\0', 'c', 'b', 'a' };
string stringCPP{ 'a', 'b', 'c', '\0', 'c', 'b', 'a'};

cout << "  |" << stringC << "|" << endl;
cout << "  |" << stringCPP << "|" << endl;
```

The output is:

```
|abc|
|abc cba|
```

Figure 24

We see that the characters of the C-string are accepted only up to the termination character; that is, the program understands the 'C-string' as a string of 3 characters: abc.

In a C++ string, the "\0" character is the same string character as all other characters, and it is displayed like a blank display.

The string length can be determined using the methods 'size()' or 'length()'.

```
cout << text.size();
int len = name.length();
```

Access to particular string characters is carried out using the operator [], as to the elements of an array.

```
char ch = name[3];

for (int i = 0; i < name.length(); i++)
{
    cout << name[i];
}

for (auto c : name)
{
    cout << c;
}
```

5.4.2. Input and output

The string type output is no different from output of the built-in data types.

```
string text{ "Hello" };

cout << text << endl;
```

Use the 'endl' manipulator to complete the string output.

Entering string with spaces and 'cin' is not required because the input is stopped after getting the first whitespace

character. Whitespace characters include such characters: space ‘ ‘, tab ‘\t’, newline ‘\n’, and ‘\v’, ‘\r’, ‘\f’.

You should to use the ‘getline()’ function to enter a string with whitespace characters. Syntax of this function:

```
getline(istream, str, delim);
```

where ‘istream’ is a stream object; ‘cin’ is used for input from the keyboard; ‘str’ is an object of the ‘string’ class, where the string will be placed; ‘delim’ is a character delimiter defines where the text ends in the string. The ‘delim’ is optional argument. And the ‘\n’ is used by default.

String input example:

```
string text;
getline(cin, text);
```

5.4.3. Operators

The ‘string’ contains the following operators:

- assignments — =;
- input-output — <<, >>;
- concatenation (string addition) — +, +=;
- comparison — ==, !=, <, >, <=, >=;
- index operators — [].

Consider the following piece of code and the output results:

```
string s1;
string s2;
string s3;
```

```
s1 = "abc";
s2 = { 'x', 'y', 'z'};
s3 = s1;

cout << " " << s1 << " " << s2 << " " << s3 << endl;

cout << " " << (s1 == s2) << " " << (s1 != s2) << " "
     << (s1 < s2) << " " << (s1 >= s2) << endl;
cout << " " << s1 + s2 << endl;

s2 += s1;

cout << " " << s2 << endl;
```

```
abc xyz abc
0 1 1 0
abcxyz
xyzabc
```

Figure 25

Using the 'string' type variable, you can assign the values of similar variables, the C-strings values, and the values of character arrays.

Strings can be compared using "equal", "not equal", "greater than", "less than" operations.

The strings are compared in lexicographic order, i.e., as if they were located in the dictionary. The line that should be placed upper another one in the dictionary is the "less" one.

The string addition operation (concatenation) adds the string before the character after the '+' in the string.

5.4.4. The ‘string’ Class Methods

All methods of the ‘string’ class can be divided into the following groups:

- characters access methods;
- iterator methods;
- search methods;
- editing methods;
- capacity methods;
- service functions (non-class members).

5.4.5. Character Access Methods

We know that the individual characters of a string can be accessed with an index (text[i]).

In addition, you can get special access to the characters in the string using access methods.

Table 1

Method	Description
at()	Getting the specified character with checking for index out of range
front()	Getting the first character (returns a reference to a character)
back()	Getting the last character (returns a reference to a character)
data()	Returns a constant pointer to C-array in a string, where data() + i == &operator[](i) for each ‘i’ in [0, size() — 1]

at() Method

```
(1) char& at (size_t pos);
(2) const char& at (size_t pos) const;
```

The ‘at()’ method is used the same as access through the operator []. The difference is that when the index is out of the string ranges, the operator [] will stay without processing. And the ‘ at()’ method will report an error (displace an exception), making the program more error-proof.

Option 1 — full string access; option 2 — only reading.

```
string name{ "12345" };
char c2 = name.at(10);
char c1 = name[10];
```

front() Method

```
(1) char& front();
(2) const char& front() const;
```

back() Method

```
(1) char& back ();
(2) const char& back () const;
```

Obviously, the ‘front()’ and ‘back()’ methods return references to the first and last characters of the string. The options are ‘full access’ and ‘read only’.

```
string name{ "12345" };
char& beg = name.front();
char& fin = name.back();
```

data() Method

```
const char* data () const;
```

The ‘data()’ method allows you to work with the string as a C-string without its modifications.

```
string name{ "12345" };
const char* pname = name.data();
```

5.5. Iterator Methods

Iterator methods return pointers to the elements of the string.

Table 2

Method	Description
begin(), cbegin()	Returns an iterator to the first element
end(), cend()	Returns an iterator pointing to the past-the-end element in the sequence
rbegin(), rend() crbegin(), crend()	Return reverse iterators (to traverse a string in reverse order)

Application example of using:

```
string text{ "this is a c++ string" };
for (auto it = text.begin(); it < text.end(); it++)
{
    if (!isalpha(*it) && !isspace(*it))
    {
        cout << *it;
    }
}
cout << endl;
```

Figure 26

In the above code fragment, we can see all characters of the string: from the first to the last, and all non-letters and non-spaces, i.e., ++ characters, are displayed on the screen.

5.6. Search Methods

Search methods allow you to find the position of the desired string or character in the string.

Table 3

Method	Description
find()	Finds the first substring equal to the given character sequence. The search starts with the given position
rfind()	Finds the last substring equal to the given character sequence. The search starts with the given position
find_first_of()	Finds the first character in a string equal to one of the characters in the given character sequences. The search starts with the given position
find_last_of()	Finds the last character equal to one of the characters in the given character sequence. The search starts from the given position
find_first_not_of()	Finds the first character of a string that is not equal to any of the characters in the given character sequence. The search starts with the given position
find_last_not_of()	Finds the last character, which is not equal to any characters in the given character sequences. The search starts from the given position

5.6.1. find() Method

```
(1) size_t find (const string& str, size_t pos = 0) const;
(2) size_t find (const char* s, size_t pos = 0) const;
```

```
(3) size_t find (const char* s, size_t pos, size_t n) const;
(4) size_t find (char c, size_t pos = 0) const;;
```

rfind() Method

```
(1) size_t find (const string& str, size_t pos = 0) const;
(2) size_t find (const char* s, size_t pos = 0) const;
(3) size_t find (const char* s, size_t pos, size_t n) const;
(4) size_t find (char c, size_t pos = 0) const;
```

The 'find()' method returns the character position or position of the first character of the found substring. The search starts at the beginning of the string, by default. (from the character with '0'). But you can specify the character number of the string in the 'pos' parameter from which you need to start the search. The 'find' method returns the string::npos value if the substring is not found.

The third 'n' parameter, if it is included, indicates the number of characters to compare.

The 'rfind' method searches from the end of the string. If the pos parameter is included, it specifies the character number from which the search towards the beginning of the string.

Consider a piece of code and the output results:

```
string text{ "This is a c++ string" };
cout << " " << text << endl;
cout << " " << text.find("is") << endl; // 1 – find (2)
cout << " " << text.find("is", 4) << endl; // 2 – find (2)
cout << " " << text.rfind("is") << endl; // 3 – rfind (2)
cout << " " << text.rfind("is", 4) << endl; // 4 – rfind (2)
cout << " " << text.find("strong", 0, 3) << endl; // 5 – find (4)
cout << " " << text.find("strong") << endl; // 6 – find (4)
```

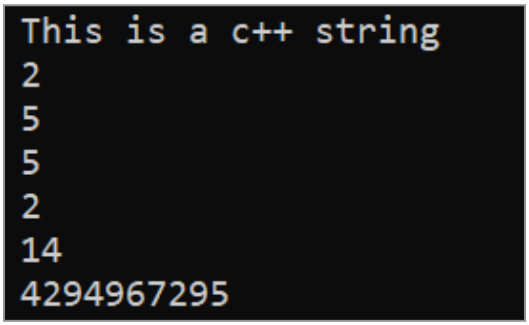


Figure 27

The first search finds "*is*" in a word "*This*" (position 2). The second search skips the word and finds "*is*" further (position 5). The third and fourth searching are executed from the end of the string to its start (5 and 2 correspondingly). The fifth search finds 3 letters of the word "*strong*" (position 14). The sixth search has not found a word "*strong*" (the value string::npos is displayed on the screen).

find_first_of() Method

```
(1) size_t find_first_of (const string& str, size_t pos = 0)
                          const;
(2) size_t find_first_of (const char* s, size_t pos = 0)
                          const;
(3) size_t find_first_of (const char* s, size_t pos, size_t n)
                          const;
(4) size_t find_first_of (char c, size_t pos = 0) const;;
```

Method searches not for a substring but any of the characters specified in this substring.

The find_last_of() method is fully similar to the find_first_of() method, but searches only from the end of the string.

Let’s look at a small example:

```
string text{ "This is a c++ string" };
cout << " " << text << endl;
cout << " " << text.find_first_of("bac") << endl;  // 1
cout << " " << text.find_last_of("bac") << endl;  // 2
```

```
This is a c++ string
8
10
```

Figure 28

The first search finds the “*a*” character (from the “*abc*” character set) — position 8. The second search (from the end) finds “c” character (from the “abc” character set) — position 10). The find_first_not_of() and find_last_not_of() methods are analogous to the methods find_first_of() and find_last_of(), but find only any from characters are not included in a substring.

Example:

```
string text{ "this is a c++ string" };
cout << " " << text << endl;
cout << " "
     << text.find_first_not_of("abcdefghijklmnopqrstuvwxyz ")
     << endl;
```

```
this is a c++ string
11
```

Figure 29

The search has returned the position of the first non-letter (i.e., character ‘+’).

5.6.2. Editing Methods

Editing methods are used for string modifications.

Table 4

Method	Description
assign()	Replaces a string content
clear()	Removes all characters from a string
erase()	Removes the specified characters from the string
insert()	Inserts characters into a string
push_back()	Adds a character to the end of a string
pop_back()	Deletes the last character of a string
append()	Adds characters to the end of a string
replace()	Replaces part of a string with the specified substring
substr()	Returns a substring
copy()	Copies a substring
resize()	Changes a string size
swap()	Replaces a string content with another string

assign() Method

```
(1) string& assign (const string& str);
(2) string& assign (const string& str, size_t subpos,
                    size_t sublen);
(3) string& assign (const char* s);
(4) string& assign (const char* s, size_t n);
(5) string& assign (size_t n, char c);
```

If you apply the assign(), the string will be replaced with:

1. The 'str' string;

2. A substring of the ' str' string (' subpos' is indicated as the starting position, and 'sublen' is indicated as the number of substring characters);
3. The 's' C-string;
4. A substring of 's' C-string (' n' indicates the number of the substring characters);
5. A given number of 'n' repetitions of the specified 'c' character.

Examples:

```
string text{ "this is a c++ string" };
string str{ "next string" };
cout << " " << text.assign(str) << endl;
cout << " " << text.assign(str, 3, 4) << endl;
cout << " " << text.assign("abcdef") << endl;
cout << " " << text.assign("abcdef", 2) << endl;
cout << " " << text.assign(5, 'a') << endl;
```

```
next string
t st
abcdef
ab
aaaaa
```

Figure 30

clear() Method

```
void clear ();
```

Erases a string content, which becomes an empty string with the length of 0.

erase() Method

```
(1) string& erase (size_t pos = 0, size_t len = npos);
(2) iterator erase (const_iterator p);
(3) iterator erase (const_iterator first, const_iterator last);
```

Erases part of the string:

1. Specified number of characters ('len') from the specified position ('pos'). The entire string is deleted by default;
2. Erases a character pointed to by the 'p' iterator;
3. Erases characters in the specified range of the iterators 'first - last'.

Examples:

```
string text{ "this is a c++ string" };
string str = text;
cout << " " << "|" << str.erase() << "|" << endl;

str = text;
cout << " " << "|" << str.erase(3, 10) << "|" << endl;

str = text;
str.erase(str.begin() + 1);
cout << " " << "|" << str << "|" << endl;

str = text;
str.erase(str.begin() + 1, str.end() - 3);
cout << " " << "|" << str << "|" << endl;
```

```
||
|thi string|
|tis is a c++ string|
|ting|
```

Figure 31

insert() Method

```
(1) string& insert (size_t pos, const string& str);
(2) string& insert (size_t pos, const string& str,
                    size_t subpos, size_t sublen = npos);
(3) string& insert (size_t pos, const char* s);
(4) string& insert (size_t pos, const char* s, size_t n);
(5) string& insert (size_t pos,   size_t n, char c);
(6) iterator insert (const_iterator p, char c);
```

Inserts in a string from the specified position ‘pos’:

1) given ‘str’ string;
2) a substring of a ‘str’ string (the starting ‘subpos’ position and the number of characters of the ‘sublen’ substring are indicated);
3) ‘s’ C-string;
4) a substring of the ‘s’ C-string (the number of characters of the ‘n’ substring is indicated);
5) given a number of ‘n’ repetitions of the specified ‘c’ character.
6) specified ‘c’ character to the position by ‘p’ iterator.

Examples:

```
string text{ "this is a c++ string" };
string theStr{ "abcdef" };

string str = text;
cout << " " <<str.insert(5, theStr) << endl;

str = text;
cout << " " << str.insert(5, theStr, 2, 3) << endl;

str = text;
```

```
str.insert(str.begin() + 2, 'A');
cout << " " << str << endl;
```

```
this abcdefis a c++ string
this cdeis a c++ string
thAis is a c++ string
```

Figure 32

push_back() Method

```
void push_back(char c);
```

Adds the 'c' character after the last character of a string.

```
text.push_back('F');
```

pop_back() Method

```
void pop_back();
```

Removes the last character of a string. If the string is empty, the function condition will be undefined, without any appeared exceptions.

```
text.pop_back();
```

append() Method

```
(1) string& append (const string& str);
(2) string& append (const string& str, size_t subpos,
                    size_t sublen = npos);
```

```
(3) string& append (const char* s);
(4) string& append (const char* s, size_t n);
(5) string& append (size_t n, char c);
```

Appends the specified characters to the end of the string.

Overloading of the 'append' method is similar to the overloading of the 'insert' method.

replace() Method

```
(1) string& replace (size_t pos, size_t len,
                     const string& str);
    string& replace (const_iterator i1, const_iterator i2,
                     const string& str);
(2) string& replace (size_t pos, size_t len,
                     const string& str, size_t subpos,
                     size_t sublen = npos);
(3) string& replace (size_t pos, size_t len, const char* s);
    string& replace (const_iterator i1, const_iterator i2,
                     const char* s);
(4) string& replace (size_t pos, size_t len,
                     const char* s, size_t n);
    string& replace (const_iterator i1, const_iterator i2,
                     const char* s, size_t n);
(5) string& replace (size_t pos, size_t len, size_t n,
                     char c);
    string& replace (const_iterator i1, const_iterator i2,
                     size_t n, char c);
```

Replaces a part of the string with the given '.str' string.

A part of the string to be replaced is pointed with:

- the starting position and length ('pos' and 'len'),
- the 'i1' and 'i2' pointers (iterators) at the substring's beginning and end.

The replacement string could be:

1) as a 'str' string;
2) as a 'str' substring (from the 'subpos' position, with the length of 'sublen');
3) as a *s C-string;
4) as a substring of a *s C-string with the length of 'n';
5) as a string created with the 'c' character, repeated 'n' times.

Examples:

```
//            01234567890123456789
string text{ "this is a c++ string" };
string str{ "abcdef" };

string txt = text;
txt.replace(10, 3, str);
cout << " " << txt << endl;

txt = text;
txt.replace(10, 3, str, 2, 2);
cout << " " << txt << endl;

txt = text;
txt.replace(10, 3, "new");
cout << " " << txt << endl;

txt = text;
txt.replace(10, 3, "1234", 1, 2);
cout << " " << txt << endl;

txt = text;
txt.replace(10, 3, 3, '!');
cout << " " << txt << endl;

txt = text;
txt.replace(txt.begin() + 10, txt.end() - 7, str);
cout << " " << txt << endl;
```

```
this is a abcdef string
this is a cd string
this is a new string
this is a 23 string
this is a !!! string
this is a abcdef string
```

Figure 33

substr() Method

```
string substr (size_t pos = 0, size_t len = npos) const;
```

Returns a new string which is a substring of this string. The substring is set with the starting position and length (‘pos’ and ‘len’). If length is not specified, the substring is set up to the end of the string.

And upon that, a source string is not modified.

Examples:

```
// 01234567890123456789
string text{ "this is a c++ string" };

cout << " " << text.substr(5, 9) << endl;
cout << " " << text.substr(5) << endl;
cout << " " << text << endl;
```

```
is a c++
is a c++ string
this is a c++ string
```

Figure 34

copy() Method

```
size_t copy (char* s, size_t len, size_t pos = 0) const;
```

It copies a substring of the given string from the 'pos' position of the 'len' length into a '*s' character array. At this, the C-string termination character ('\0') is not copied.

Example:

```
// 01234567890123456789
string text{ "this is a c++ string" };

char carr[10]{ 0 };
text.copy(carr, 8, 5);
cout << " " << carr << endl;
```

```
is a c++
```

Figure 35

resize() Method

```
(1) void resize (size_t n);
(2) void resize (size_t n, char c);
```

It sets the string size equal to the given 'n' number. If 'n' is less than the current size, the string will be cut to the set size. If 'n' is greater than the current size, the string will be increased to the set size. However, if the 'c' character is set, the string is will be filled up with the required amount of 'c'. And, if 'c' is not set, the string will be filled up with zeros ('\0').

Example:

```
string str{ "abcdef" };
str.resize(10, '*');
cout << " " << str << endl;
```

```
abcdef****
```

Figure 36

swap() Method

```
void swap (string& str);
```

It swaps one string with another one.

Example:

```
string text{ "this is a c++ string" };
string str{ "abcdef" };
text.swap(str);
cout << " " << text << endl;
cout << " " << str << endl;
```

```
abcdef
this is a c++ string
```

Figure 37

5.6.3. Capacity Methods

In addition to length, strings have one more characteristic of ‘size’ is a capacity.

When creating and changing a string, it is allocated memory not exactly by the number of characters that will fill

this string, but more than enough. This is done to decrease the number of dynamic memory allocation requests while slightly increasing the string size.

Capacity methods change length and strings' capacity without their content modifications.

Table 5

Method	Description
empty()	Checks if a string is empty
size(), length()	Returns the number of characters in a string
max_size()	Returns the maximum number of elements a string contains
reserve()	Sets the string capacity, minimum of size(). New storage memory is allocated as needed
capacity()	Returns the number of characters for which a string has an allocated space
shrink_to_fit()	Request for a capacity reduction from 'capacity()' to 'size()'

empty() Method

```
bool empty() const;
```

It will return 'true' if the string is empty (string length is 0).

```
if (str.empty())
{
    cout << "No content" << endl;
}
```

size() Method

```
size_t size() const;
```

length() Method

```
bool length() const;
```

These two methods return the length of the string (the number of characters in the string).

capacity() Method

```
size_t capacity() const;
```

It returns the capacity of the string — the number of bytes allocated for the string.

max_size() Method

```
size_t max_size() const;
```

Returns the maximum possible length of a string.

reserve() Method

```
void reserve (size_t n = 0);
```

The function sets the new string capacity 'n'. If 'n' is greater than the current capacity, the capacity increases to 'n' or more. If 'n' is less than the current capacity, then the reduction is performed so that the length of the string remains the same.

shrink_to_fit() Method

```
void shrink_to_fit();
```

The function reduces the string capacity, making it equal to a string length. In fact, capacity after the function execution can remain greater than the string's length for memory optimization.

Examples:

```
string text{ "this is a c++ string" };
string str{ "abcdef" };
string none;
cout << " " << text.length() << endl;
cout << " " << text.size() << endl;
cout << " " << text.capacity() << endl;
cout << " " << text.max_size() << endl;
cout << " " << str.length() << endl;
cout << " " << str.capacity() << endl;
cout << " " << str.max_size() << endl;
cout << " " << none.length() << endl;
cout << " " << none.capacity() << endl;
text.reserve(8);
cout << text << endl;
cout << " " << text.capacity() << endl;
text.shrink_to_fit();
cout << " " << text.capacity() << endl;
```

```
 20
 20
 31
 2147483647
 6
 15
 2147483647
 0
 15
this is a c++ string
 31
 31
```

Figure 38

5.6.4. Service Functions (Non-Class Members)

Use service functions for string data conversions and for reading strings with whitespace characters.

Table 6

Method	Description
getline()	Reads data from the stream up to the delimiter or to the character limit
stoi(), stol(), stoll()	Converts a string to an integer (int), long integer (long), “extra-long” integer (long long)
stoul(), stoull()	Converts a string to an unsigned long integer (long, long long)
stof(), stod(), stold()	Converts a string to a floating-point number (float, double, long double)
to_string()	Converts an integer or float to a string

getline() Method

```
(1) istream& getline (istream& is, string& str, char delim);

(2) istream& getline (istream& is, string& str);
```

Reads a string from ‘is’ input stream into a ‘str’ string variable. Reading is implemented up to the delimiter character ‘delim’ in the input stream. If delimiter is not specified by default, delimiter — ‘\n’ (a ‘newline’ character).

Example:

```
string text;
getline(cin, text);
```

Methods: stoi(), stol(), stoll()

```
int stoi (const string&  str, size_t* idx = 0,
          int base = 10);

long stol (const string&  str, size_t* idx = 0,
          int base = 10);

long long stoll (const string&  str, size_t* idx = 0,
          int base = 10);
```

These functions convert the 'str' string into an integer.

If the 'idx' pointer is not equal to zero, the function will place the position in the string at this reference that corresponds to the first non-digit character.

The 'base' parameter defines the standard decimal number system by default. If necessary, you can specify a different number system.

Examples:

```
string text1{ "129" };
int number1 = stoi(text1);
cout << " " << number1 << endl;

unsigned int pos;
string text2{ "124,8" };
int number2 = stoi(text2, &pos);
cout << " " << number2 << endl;
cout << " " << pos << endl;

string text3{ "1248" };
int number3 = stoi(text3, &pos, 8);
cout << " " << number3 << endl;
cout << " " << pos << endl;
```

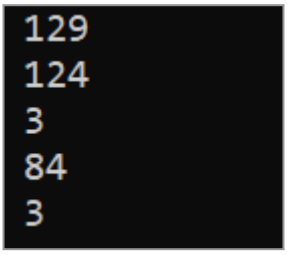


Figure 39

In the first example, the string “129” is converted to the number 129.

In the second example, the symbol with the number 3 (comma) in the string “*124.8*” is no longer a digit. So, the resulting number 124 is displayed on the screen, and the position where the number ends is 3.

In the third example, parameter 8 specifies that the number in the octal number system will be processed. Therefore, the number 8 is not a number by now (not an octal digit), but 124 in octal represents the decimal number 84 (1 * 64 + 2 * 8 + 4 = 84).

What we see on the screen.

Methods: stoul(), stoll()

```
unsigned long stoul (const string& str, size_t* idx = 0,
                     int base = 10);

unsigned long long stoull (const string&  str,
                     size_t* idx = 0, int base = 10);
```

These functions will convert the ‘str’ string into positive (unsigned) integers. Function parameters are similar to the reviewed above.

Example:

```
unsigned int number1 = stoul(text1);
unsigned long number2 = stoul(text2);
```

Methods: stof(), stod(), stold()

```
float stof (const string&  str, size_t* idx = 0);
double stod (const string&  str, size_t* idx = 0);
long double stold (const string&  str, size_t* idx = 0);
```

These functions will convert the 'str' string into floats. The function parameters are similar to the reviewed above.

Example:

```
string orbits("365.24 29.53");
unsigned int sz;

double earthTurn = stod(orbits, &sz);
double moonTurn = stod(orbits.substr(sz));
cout << "The moon completes " << (earthTurn / moonTurn)
     << " orbits per Earth year" << endl;
```

```
The moon completes 12.3684 orbits per Earth year
```

Figure 40

In the above example, the first part of the string "*365.24 29.53*" is first converted to a floating-point number. Then, the second part (a substring from the end of the first part to the end of the string). And after then, it calculates times the Moon turns round the Earth in a year.

to_string() Method

```
(1) string to_string (int val);
(2) string to_string (long val);
(3) string to_string (long long val);
(4) string to_string (unsigned val);
(5) string to_string (unsigned long val);
(6) string to_string (unsigned long long val);
(7) string to_string (float val);
(8) string to_string (double val);
(9) string to_string (long double val);
```

It converts numbers in different representations — to a string.

Example:

```
string piIs = "Pi is " + to_string(3.1415926);
cout << " " << piIs << endl;
```

```
Pi is 3.141593
```

Figure 41

5.7. Examples of Applying the ‘string’ Class

5.7.1. Text search program

The program code is in the folder *Lesson05\les05_15*. The ‘text’ variable in the ‘main’ function is initialized with a strophe from one of the Kipling’s poem. This text is looking for a word “and” (substring) — first case-sensitive and then case-insensitive.

```
#include <iostream>
#include <iomanip>
#include <conio.h>
using namespace std;

template <typename T>
void display(T* items, int size);
int countAll(const string where, const string what);
int* findAll(string where, string what, int& count);
string toUpr(const string text);
int* findAllR(const string where, const string what,
              int& count);

int main()
{
    string text
    { R"(
        If you can keep your head when all about you
        Are losing theirsand blaming it on you;
        If you can trust yourself when all men doubt you,
        But make allowance for their doubting too:
        If you can wait and not be tired by waiting,
        Or, being lied about, don't deal in lies,
        Or being hated don't give way to hating,
        And yet don't look too good, nor talk too wise;
    )" };

    cout << "In text" << endl;
    cout << text << endl;

    cout << "Search for word \"and\"" << endl;
    cout << endl;

    cout << "case sensitive" << endl;
    int count;
    int* pos = findAll(text, "and", count);
    display(pos, count);
```

```
    cout << endl;

    cout << "case insensitive" << endl;
    pos = findAllR(text, "and", count);
    display(pos, count);

    _getch();
    return 0;
}
```

The program result is shown in Figure 42.

```
In text

If you can keep your head when all about you
Are losing theirsand blaming it on you;
If you can trust yourself when all men doubt you,
But make allowance for their doubting too :
If you can wait and not be tired by waiting,
Or, being lied about, don't deal in lies,
Or being hated don't give way to hating,
And yet don't look too good, nor talk too wise;

Search for word "and"

case sensitive
      63     196

case insensitive
      63     196     308
```

Figure 42. Finding a substring in the text

Lists of all found words on their positions are displayed on the screen.

In the case-sensitive search, this is the end of the word “their sand” in the second line and the word “and” in the fifth line.

In a case-insensitive search, the word “And” is on the last line.

The search is performed by the ‘ findAll’ (case-sensitive search) and the ‘ findAllR’ functions (case-insensitive search). Both of these functions return a pointer to an array of found the int* pos positions, when that array is displayed on the screen by the ‘ display’ function.

The ‘findAll’ function gets the following parameters: where to look for — ‘string where’, what to look for — ‘string what’, and returns a pointer to an array of found positions and the number of found positions due to the int& count.

```
int* findAll(string where, string what, int& count)
{
    count = countAll(where, what);

    if (count == 0)
    {
        return nullptr;
    }

    int* array = new int[count];
    int pos = -1;

    for (int i = 0; i < count; i++)
    {
        pos = where.find(what, pos + 1);
        array[i] = pos;
    }

    return array;
}
```

The function first determines the number of found positions by means of the ‘countAll’ function, then it creates an array of the desired size and searches for the word in a loop.

Each time the search resumes from the position following the one found (pos + 1). -1 is set for the first search with 0 ‘pos’ position. If there are no search words in the text, the function returns ‘nullptr’.

The ‘countAll’ function counts the number of search word occurs in the text

```
int countAll(const string where, const string what)
{
    int count = 0;

    int pos = where.find(what);
    while (pos != string::npos)
    {
        count++;
        pos = where.find(what, pos + 1);
    }
    return count;
}
```

The search starts from the beginning of the text, and it is performed from the next position every time.

The ‘display’ function template is used to display the found positions of the word in the text.

```
template <typename T>
void display(T* items, int size)
{
    for (int i = 0; i < size; i++)
    {
```

```
        cout << setw(8) << items[i];
    }
    cout << endl;
}
```

The 'findAlRl' function has the same parameters as the 'findAll', and it uses the 'findAll' for searching.

```
int* findAllR(const string where, const string what,
              int& count)
{
    return findAll(toUpr(where), toUpr(what), count);
}
```

At this, both the text, which you want to use the search, and the word, you are looking for, are converted to uppercase due to the 'toUpr' function. Thus, case differences are discarded. The 'toUpr' function takes the source string and returns it in uppercase.

```
string toUpr(const string text)
{
    string uText = text;

    for (int i = 0; i < uText.size(); i++)
    {
        uText[i] = toupper(uText[i]);
    }
    return uText;
}
```

A copy of the original string is created, and every character is converted to upper case.

5.7.2. Program for Creating a Text Frequency Dictionary

The program code is in the folder *Lesson05\les05_16*.

The ‘main’ function initializes the ‘text’ variable and creates a dictionary for this text, indicating repetitions of each word. Obviously, to create a real dictionary, the program must be more complicated and understand the grammar. But we will not take into account these nuances in the first approximation.

```
#include <iostream>
#include <iomanip>
#include <conio.h>
#include <vector>
#include <algorithm>
using namespace std;

struct Entry
{
    string word;
    int count;
};

vector<Entry> getDictionary(string text);
vector<string> getWords(string text);

int main()
{
    string text
    { R"(
        If you can keep your head when all about you
        Are losing theirsand blaming it on you;
        If you can trust yourself when all men doubt you,
        But make allowance for their doubting too:
        If you can wait and not be tired by waiting,
        Or, being lied about, don't deal in lies,
```

```
        Or being hated don't give way to hating,
        And yet don't look too good, nor talk too wise;
    )" };

    cout << "From text" << endl;
    cout << text << endl;
    cout << "Create dictionary" << endl;
    cout << endl;

    vector<Entry> dictionary = getDictionary(text);
    for (auto entry : dictionary)
    {
        cout << setw(12) << entry.word << setw(4)
             << entry.count << endl;
    }
    cout << endl;

    _getch();
    return 0;
}
```

The frequency dictionary is an array of the 'Entry' structures (word, number of repetitions).

The dictionary is created with the 'getDictionary' function.

```
vector<Entry> getDictionary(string text)
{
    vector<Entry> entries;
    vector<string> words = getWords(text);
    sort(words.begin(), words.begin() + words.size(),
         sortByWords);
    int i = 0;

    while (i < words.size())
    {
        Entry entry{ words[i], 0 };
```

```
        while (i < words.size() && words[i] == entry.word)
        {
            entry.count++;
            i++;
        }
        entries.push_back(entry);
    }

    return entries;
}
```

This function creates a vector of all the words in the text (vector<string> words) through the ‘getWords’ function, sorts alphabetically, and then it rewrites into a vector of the dictionary (vector<Entry> entries). In this case, each word is rewritten only once, and the number of repetitions of the word is entered.

Word vector sort is implemented by the ‘sort’ library function, where the word comparison function is passed in as a ‘sortByWords’ parameter.

```
bool sortByWords(const string& word1, const string& word2)
{
    return word1 < word2;
}
```

The ‘getWords’ function forms a set of all words included in the text.

```
vector<string> getWords(string text)
{
    vector<string> words;
    int pos = 0;
```

```
    while (pos < text.size())
    {
        while (pos < text.size() && !isalpha(text[pos]))
        {
            pos++;
        }

        if (pos == text.size())
        {
            break;
        }
        int wordStart = pos;

        while (pos < text.size() && isalpha(text[pos]))
        {
            pos++;
        }
        int wordLen = pos - wordStart;
        words.push_back(text.substr(wordStart, wordLen));
    }

    return words;
}
```

The 'pos' variable contains the current position of the text and iterates through all the characters in the text. If a loop that is limited by the size of the text (pos < text.size):

- it includes the beginning of the next word (or the end of the text); at this, all characters are missed, but not letters;
- the end of the word — exit;
- the beginning of a word is remembered;
- includes the end of a word or the end of a text, and upon that, all letters are missed;
- a word is formed and added to the vector of words.

The generated word vector is returned to the calling function. The snippet with the results is shown in Figure 43.

```
From text

If you can keep your head when all about you
Are losing theirsand blaming it on you;
If you can trust yourself when all men doubt you,
But make allowance for their doubting too :
If you can wait and not be tired by waiting,
Or, being lied about, don't deal in lies,
Or being hated don't give way to hating,
And yet don't look too good, nor talk too wise;

Create dictionary

         And   1
         Are   1
         But   1
          If   3
          Or   2
       about   2
         all   2
   allowance   1
         and   1
          be   1
       being   2
     blaming   1
          by   1
         can   3
        deal   1
```

Figure 43. Creating a Text Frequency Dictionary

It would be possible to convert all text to lowercase in advance (see the previous program), then the results were more accurate.

6. Summary

Section 1. Static Polymorphism and Templates as a Special Type

- Polymorphic code processes different types of values while running a program.
- There are two types of polymorphism:
  - runtime polymorphism (or dynamic polymorphism);
  - compile-time polymorphism (or static polymorphism).
- Dynamic polymorphism is implemented at runtime. It is the use of virtual functions.
- Static polymorphism is implemented at compile time. C++ provides three kinds of static polymorphism:
  - function overloading;
  - operator overloading;
  - templates. There are function templates and class templates.
- The type parameter compiler determines for what type the function should be generated. But in some cases, ambiguity may arise, and then it will be necessary to indicate a type parameter when calling a template function is necessary.
- The function template has an important feature: until there is no function call, a template does not compile. Template compiles to the new version of the function for each type parameter for which it is called.

- The function template body must be placed in the header file (in the H-file, not the CPP-file).
- Polymorphism usage in all its varieties makes programs understandable and easier to maintain. In addition to this, applying C++ templates leads to shorter and easier modifiable programs.
- Templates are widely used in the standard C++ libraries, and it is hard to see writing modern programs without intensive use of these libraries.

Section 2. Class Templates

- In C++, generic programming is based on function- and class templates.
- Class templates allow you to create a description of an abstract class. While using the template, it is expanded by the compiler into definite classes that implement the specified functions for various data types.
- The class template starts with 'template' followed by a list of class template parameters in angle brackets.
- Class template parameters can be type parameters, regular type parameters, and template parameters.
- The template class code has no difference from a regular class: class fields, constructor, class methods. But only one difference is that instead of pointing to a specific data type, the name of the template type parameter is used (T) in some parts of the code.
- Be sure to specify the template type parameter in angle brackets when calling the class constructor:

```
Array<int> intArray;
```

- And call the necessary methods as a next step.

```
intArray.display();
```

- Method implementation outside the class body requires a detailed description of the types used in the method.

```
template<class T>
T Sample<T>::square(T number)
{ ...
```

- template<class T> and Sample<T> must be obligatory indicated. All class template code (including class declaration and implementation of its methods) is recommended to be placed in one header file.
- Template parameters can be the non-type parameters as well, for example:

```
template <class T, int size>
class StatArray
```

- Class templates can be as the template parameters, for example:

```
template<template<class> class T, class T1>
struct Some
```

- Class template parameters, like the standard parameters, can be default parameters, for instance:

```
template <class T = int, int size = 10>
class Array
```

- The template code can have certain requirements to the used types parameters. The template code should be overloaded by the operators and functions of the used type.
- Class templates require the following elements in the types they use:
  - ▷ default constructor;
  - ▷ copy constructor;
  - ▷ assignment operator;
  - ▷ comparison operators;
  - ▷ input/output operators.
- Lots of ready-to-use templates (classes and functions) are contained in the Standard Template Library (STL). This library is an integral part of C++.
- One of the most used STL templates is a ' vector' (one-dimensional array) — a dynamic array template.
- A fundamental definition of a function template for a particular data type is called template specialization. Template specialization exists to modify the condition of the general template to the specifics of individual data types.
- Function template specialization is a definition where the ' template' keyword follows by empty angle brackets <>, and they are followed by the definition of a specialized template which includes the name of the template, the arguments for its specializing, the list of function parameters, and its body:

```
template<>returnDataType functionName<dataType>
        (argumentList){functionBody}
```

- The template specialization must be placed after the general template.
- If a specialization defines the inline function, its definition must be in a header file.
- If specialization determines not an inline function, the header file should contain only its declaring (prototype). Defining (function body) must be in one of the implementation files.
- Full (explicit) specialization of class templates is a defining a class template where the empty angle brackets <> follow after the word 'template', followed by the name of the class, definite type parameters of the class, and the class defining:

```
template <> className<dataTypeList>{ classDefinition }
```

- If a class template has multiple parameters, then you can specialize it only for one or more arguments, leaving the others unspecialized, in other words — to allow setting their values when using the template. Such specialization is called a partial class template specialization.

Section 3. Variadic Templates

- In C++, there are functions with a variable number of parameters. Such functions are declared in the same way as ordinary functions, but instead of the last ellipsis is put in the first parameter:

```
return_type function_name(parameter_list, ...)
```

- C++ has let to define templates that take a variable number of parameters. A template with a variable number of parameters (Variadic Template) is a template with an unknown number of parameters. Variadic template syntax:

```
template<class... Types>
```

where 'Types' is a type pack. A type package can contain type- and non-type parameters.

- A type pack can be used in a function's parameter list:

```
template<class... Types>
void func(Types... parameters)
```

where 'parameters' is a pack of parameters whose types are specified by the specified 'Types' type packs, a parameter pack can only be in a template function, and it must match the type pack.

- You can use both a type and parameter pack inside a function. To do this, we put an ellipsis after the pack name . . ., which is interpreted by the compiler as expanding the pack into a comma-separated list at the specified position. The common way of defining the variadic template functions is recursive defining.

Section 4. Applying std::initializer_list

- For static and dynamic arrays initialization it is convenient to use an initialization list:

```
int array[] { 7, 6, 5, 4, 3, 2, 1 };
```

- But the default initialization list doesn't work in generated classes. The C++ compiler (as of C++11) automatically converts the initialization list into an object of std::initializer_list type. Therefore, for that, a class object could be initialized with a list; it will be enough to create a constructor in this class. Such a constructor will take an object of the std::initializer_list, type as a parameter, for e.g.

```
IntArray(const std::initializer_list<int>& list)
```

- For working on the std::initializer_list, you need to assign the header file <initializer_list>
- Likewise, you can add to a class template an option to use an initialization list.

Section 5. The 'string' Class

- ASCII is a 7-bit character encoding, symbols, the Latin alphabet characters, numbers, and punctuation symbols with codes from 0 to 127.
- Many 8-bit encodings have been created for various national alphabets, in which the first 127 characters match with ASCII. And the second part of the chart (from 128 to 255) included symbols of other alphabets.
- Unicode is a character code chart. It consists of 1,114,112 items grouped into 17 blocks of 65536 characters. The zero block is the main one, and it contains the symbols of all modern alphabets.
- Based on Unicode, there are the following encodings: UTF-8, UTF-16, UTF-32. They differ in the number of bytes allocated for the code of one character.

- C++ has the following (built-in) data types for storing character code:

```
char, char8_t, char16_t, char32_t
```

- C++ contains different types of strings to store data represented by different types of characters: 'string' with the 'char' type parameters , 'wstring' with the 'wchar_t' type parameters; 'u16string' with the 'char16_t' type parameters; 'u32string' with the 'char32_t' type parameters.
- Advantages of the 'string' class compared to C-strings:
  - the 'string' class object is a dynamic array, so it is easy to modify;
  - work on this object is protected from addressing errors;
  - for working with the 'string' objects, the class has a large set of suitable functions;
  - the 'string' class is a container. So, you can use the full set of functions and operations common to all containers for working on the 'string' objects.
- Uninitialized 'string' object gets an empty ("") value. You can initialize strings:
  - with a constant array of characters:

```
string text{ "Hello" };
```

  - with repetition of characters (in brackets!)

```
string name( 8, 'a' );
```

  - with using the assignment operator

```
string month = "March";
```

- << is used to output the strings. You need to use the input operator >> and istream getline(istreaminstream, stringstr, chardelim) function to enter strings.
- The 'string' contains the following operators:
  - ▷ assignments — =,
  - ▷ input-output — <<, >>,
  - ▷ concatenation (string addition) — +, +=,
  - ▷ comparison — ==, !=, <, >, <=, >=,
  - ▷ index operators — []
- Character access methods:

at()	Getting the specified character with checking for index out of range
front()	Getting the first character (returns a reference to a character)
back()	Getting the last character (returns a reference to a character)
data()	Returns a constant pointer to C-array in a string, where data() + i == &operator[](i) for each 'i' in [0, size() − 1]

- Iterator Methods

begin(), cbegin()	Returns an iterator to the first element
end(), cend()	Returns an iterator pointing to the past-the-end element in the sequence
rbegin(), rend(), crbegin(), crend()	Return reverse iterators (to traverse a string in reverse order)

- Search Methods:

find()	Finds the first substring equal to the given character sequence. The search starts with the given position

rfind()	Finds the last substring equal to the given character sequence. The search starts with the given position
find_first_of()	Finds the first character in a string equal to one of the characters in the given character sequences. The search starts with the given position
find_last_of()	Finds the last character equal to one of the characters in the given character sequence. The search starts from the given position
find_first_not_of()	Finds the first character of a string which is not equal to any of the characters in the given character sequence. The search starts with the given position
find_last_not_of()	Finds the last character which is not equal to any of the characters in the given character sequences. The search starts from the given position

- Editing Methods

assign()	Replaces a string content
clear()	Removes all characters from a string
insert()	Inserts characters into a string
erase()	Removes the specified characters from the string
push_back()	Adds a character to the end of a string
pop_back()	Deletes the last character of a string
append()	Adds characters to the end of a string
replace()	Replaces part of a string with the specified substring
substr()	Returns a substring
copy()	Copies a substring
resize()	Changes a string size
swap()	Replaces a string content with another string

- Capacity Methods

empty()	Checks if a string is empty
size() length()	Returns the number of characters in a string

max_size()	Returns the maximum number of elements a string contains
reserve()	Sets the string capacity, minimum of size(). New storage memory is allocated as needed
capacity()	Returns the number of characters for which a string has an allocated space
shrink_to_fit()	Request for a capacity reduction from 'capacity()' to 'size()'

- Service Functions (Non-Class Members)

getline()	Reads data from the stream up to the delimiter or up to the character limit
stoi() stol() stoll()	Converts a string to an integer (int), long integer (long), "extra-long" integer (long long)
stoul() stoull()	Converts a string to an unsigned long integer (long, long long)
stof() stod() stold()	Converts a string to a floating-point number (float, double, long double)
to_string()	Converts an integer or float to a string

7. Homework

1. Text compression

A text line contains words separated by spaces in an arbitrary number. Text compression consists in that one space is left between words, and spaces are removed after the last word (spaces before the first word are preserved). If the string contains only spaces, then all of them are kept.

Write a function that compresses a text.

2. Removing comments

You have a program text in C++. Comments start with "//" and continue to the end of the current string, or start with "/*" and end with "*/". In the last case, comments can be in the middle of a line or placed in multiple strings.

Write a function that removes comments.

3. Search and selection

Write a function that takes two strings as parameters and returns a copy of the first parameter. All occurrences of the second parameter taken in "()".

For example, if the parameters were the strings

```
"abaracadabra" and "ab"
```

then you have to return

```
"(ab)aracad(ab)ra"
```

4. Calculation of amounts

There are several expressions in the text line: sums and differences of two natural numbers n1+n2 and n1-n2.

Write a function that replaces expressions with their numeric values.

For example, for the line

```
"alpha 5+26 beta 72-35 gamma 32+45 etc"
```

the result should be

```
"alpha 31 beta 37 gamma 77 etc"
```

5. Roman numerals

Write a function that takes a string as a record of a natural number (up to 4000) and returns a string containing the record of this number in Roman numerals.

For instance, from the line

```
"961"
```

you should get a string like this:

```
"CMLXI"
```

6. Full name (path) of the file

Write a function that gets a file path and returns:

a) Path without a file name (if there was no path, output an empty string).
b) The name of the last folder in the path.
c) File name with extension without including a path.

d) File name extension.
e) File name without path and extension.

For instance, from the line "*C:\Step\C++lesson_03\Docs\Less03.docx*" you should get the following:

а) «*C:\Step\C++lesson_03\Docs*»;
б) «*Docs*»;
в) «*Less03.docx*»;
г) «*.docx*»;
д) «*Less03*».

8. Terminology

- ASCII,
- class template,
- function template,
- std::initializer_list,
- std::string,
- STL — Standard Template Library,
- string,
- UTF-8,
- UTF-16,
- UTF-32,
- Unicode,
- template,
- variadic template,
- vector,
- dynamic polymorphism,
- type default value,
- iterator,
- generic programming,
- parameter pack,
- type pack,
- non-type parameter,
- type parameter,
- polymorphism,
- full class template specialization,
- template specialization,
- C-strings,
- static polymorphism,
- generalized programming,
- Partial Class Template Specialization,
- explicit class template specialization.

Lesson 5.
Class Templates. The ‘string’ Class